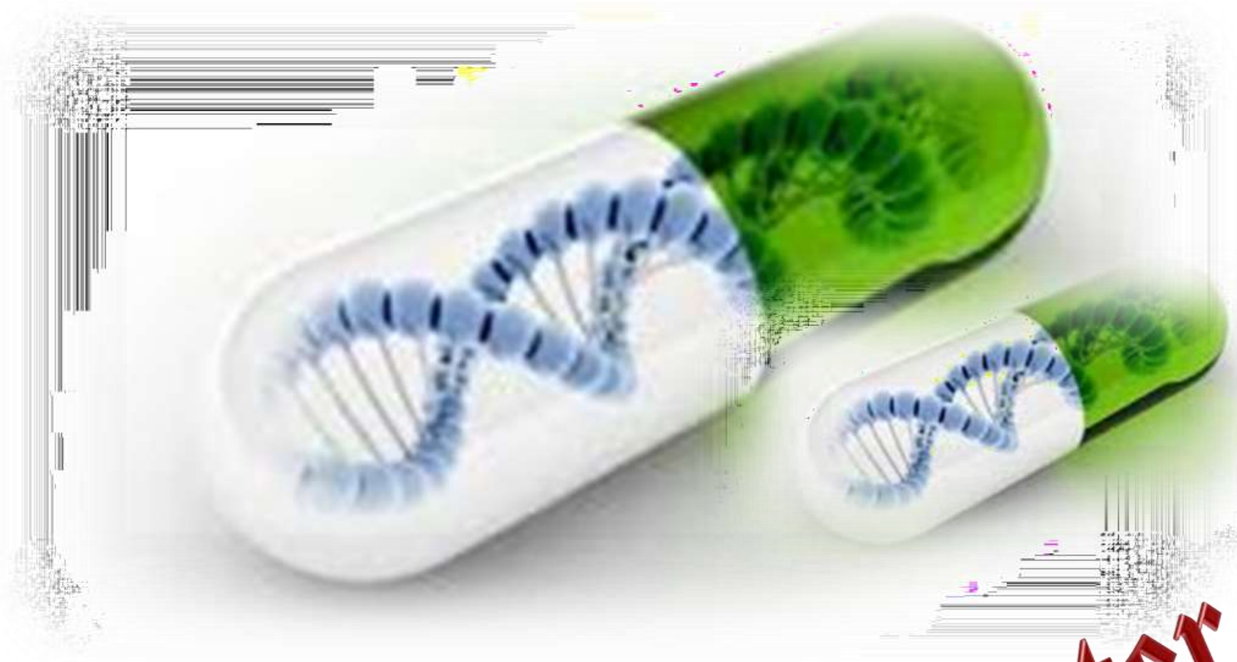


RV College of Engineering®

Go, change the world

Computer Graphics - 16CS73

By
Department of CSE,
RVCE



Computer Graphics

Objectives

- Explanation of the basic graphics system.
- Explanation of Imaging systems: Physical and synthetic.
- Working of a synthetic camera model.
- Basic idea of the programmer's interface.
- Learning the basic design of a graphics system.
- Introducing pipeline architecture.
- Generation of the ***Sierpinski gasket.***

Computer Graphics

- Computer Graphics deals with all aspects of ***creating images*** with a computer.
- Computer Graphics deals with
  - **Geometric modeling:** creating mathematical models of 2D and 3D objects.
  - **Rendering:** producing images of the objects.
  - **Animation:** representing time dependent behaviour of objects.
- Computer Graphics software – ***OpenGL.***



Open Graphics Library

- ***Tool or a method*** used to develop graphics application.
- Easy to learn.
- Popular graphics system.

Applications of Computer Graphics



Applications of Computer Graphics

- Display of information
- Design
- Simulation and animation
- User interfaces

Display of information

- CG is a *medium* to convey information among people.
- More than 4000 yrs ago, Babylonians displayed floor plans of buildings on stones.
- More than 2000 yrs ago, Greeks conveyed their architectural ideas graphically.



Display of information

- Maps can be developed and manipulated to display geographical and celestial information over the Internet.



Display of information

- Medical imaging – CT scan, MRI scan, PET scan etc generate 3-D data & is subjected to algorithmic manipulation to provide useful information.
- Fluid flow, molecular biology and mathematics.



Design

- Interactive graphics tools are extensively used in the ***field of architecture, mechanical engineering.***
- CG is used in the design of VLSI circuits, creation of ***characters for animations.***

Simulation and animation

- Once graphics system evolved to be capable of generating sophisticated images in real time, engineers and researchers began to use them as simulators.
- **Training of pilots (video)**
  - A flight simulator is a device that artificially re-creates aircraft flight and the environment in which it flies, for pilot training, design, or other purposes.
  - Training expenses have been reduced with increased safety.

Simulation and animation

Training of pilots (video)





Simulation and animation

- Designing robots, planning its path, simulating its behavior in complex environments.
- Animation in television, motion pictures and advertising industries.







Virtual Environment !





Massive multiplayer games



User interfaces

- In the field of computers, ***user interaction system*** is dominated by windows, icons, menus and pointing devices like mouse.
- This style of interface makes wide use of CG.
- GUI is one of the most popular interfaces today.

Objectives

- Development of the OpenGL API
- OpenGL as state machine
- Fundamental OpenGL primitives
- Attributes
- Simple viewing
  - Two-dimensional viewing as a special case of 3D viewing
- Interaction with the Window System
- Sierpinski Gasket Program – triangles
- 3D Sierpinski Gasket Program
- Introduce hidden-surface removal

RV College of Engineering

Why OpenGL API ?

Go, change the world

- OpenGL is not a programming language like C or C++. It is more like ***C runtime library*** which provides some prepackaged functionality.
- Specifically OpenGL allows us to Access the GPU of PC.
- Device independent, platform independent API. And It is Implemented in Graphics
- Defined as a ***software interface to graphics hardware***.
- 3D graphics and modeling library that is extremely portable and very fast.
- Using OpenGL, elegant 3D graphics can be created.
- It uses algorithms developed and optimized by Silicon Graphics Inc(SGI) , an acknowledged world leader in graphics and animation.



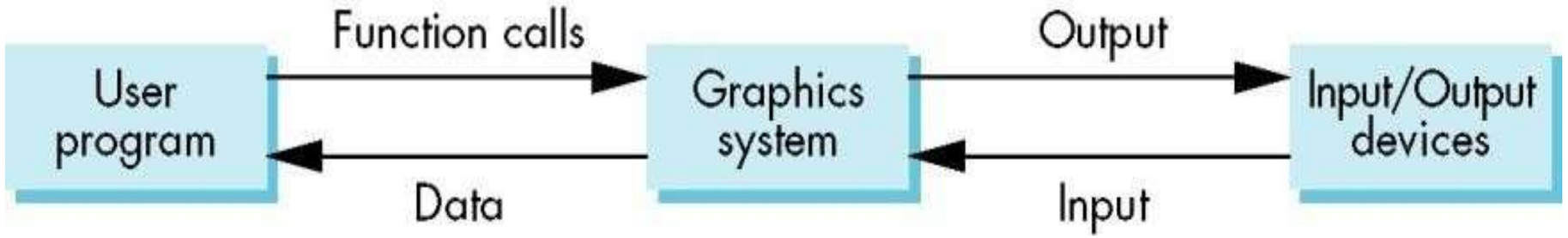
GRAPHICS FUNCTIONS

RV College of Engineering

Go, change the world

Graphics functions

- Basic model of graphics package is ***a black box.***
- ***Black box*** - a term that engineers use to denote a system whose properties are described by only its inputs and outputs; nothing is known about its internal workings.



OpenGL

RV College of Engineering

Graphics functions

Go, change the world

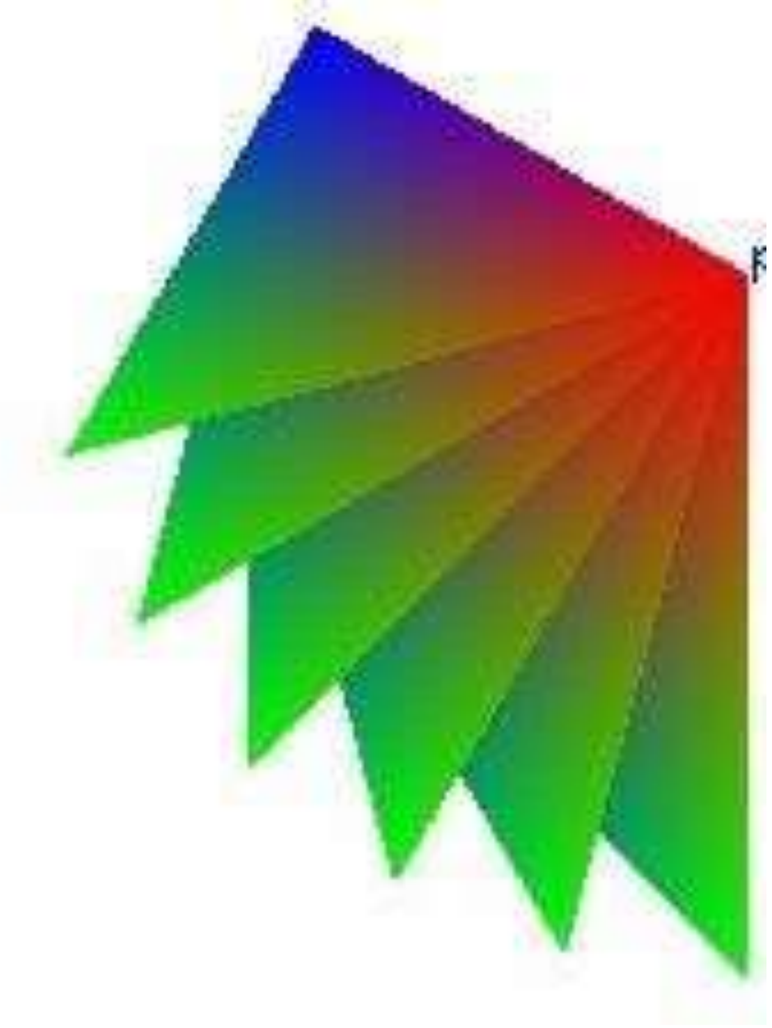


- Hence graphics system can be viewed as a box whose
  - ***inputs*** are ***function calls*** from a user program (say measurements from input devices, such as the mouse and keyboard and possibly other input, such as messages from the operating system).
  - ***outputs*** are primarily the ***graphics*** sent to the output devices.

RV College of Engineering

Go, change the world

Graphics Functions - Types



OpenGL

RV College of Engineering

Graphics Functions

Go, change the world

(June 2010, Dec 2010, July 2011, July 2013)

Graphics functions can be grouped as

1. Primitive functions
2. Attribute functions
3. Viewing functions
4. Transformation functions
5. Input functions
6. Control functions
7. Query functions

1. Primitive functions
2. Attribute functions
3. Viewing functions
4. Transformation functions
5. Input functions
6. Control functions
7. Query functions

Primitive functions enable the programmer to create some ***basic primitives*** such as points, line segments, polygons, text etc.

Example :

```
glVertex*();
glBegin();
glEnd();
```

1. Primitive functions
2. Attribute functions
3. Viewing functions
4. Transformation functions
5. Input functions
6. Control functions
7. Query functions

Example :

```
/* Draw a triangle */

glBegin (GL_LINE_LOOP);
   glVertex3f(-0.3, -0.3, 0.0);
   glVertex3f(0.0, 0.3, 0.0);
   glVertex3f(0.3, -0.3, 0.0);
glEnd();
```

1. Primitive functions
2. Attribute functions
3. Viewing functions
4. Transformation functions
5. Input functions
6. Control functions
7. Query functions

Attribute functions perform operations ranging from ***choosing the color***, to ***picking a pattern*** with which to fill the inside of a polygon, to selecting a type face for the titles on a graph.

Example :

```
glColor3f(1.0,0.0,0.0);
glClearColor(1.0,1.0,1.0,1.0);
```

1. Primitive functions
2. Attribute functions
3. Viewing functions
4. Transformation functions
5. Input functions
6. Control functions
7. Query functions

Viewing functions enable the programmer to specify ***views*** such as front view, top view, side view. Also provide ***zoom in*** and ***zoom out*** facilities.

Example :

```
glViewport();
glMatrixMode(GL_PROJECTION);
glMatrixMode(GL_MODELVIEW);
gluOrtho2D();
```

1. Primitive functions
2. Attribute functions
3. Viewing functions
4. Transformation functions
5. Input functions
6. Control functions
7. Query functions

Transformation Functions enable the programmer to carryout geometric transformations such as ***rotation, scaling, translation***.

Example :

```
glTranslatef();
glRotatef();
```

1. Primitive functions
2. Attribute functions
3. Viewing functions
4. Transformation functions
5. Input functions
6. Control functions
7. Query functions

Input functions enable the programmer to design interactive programs.

These functions can accept ***inputs from mouse, keyboard, data tablets*** etc.

Example :

```
glutMouseFunc(mouse);
```

RV College of Engineering

Go, change the world

1. Primitive functions
2. Attribute functions
3. Viewing functions
4. Transformation functions
5. Input functions
6. Control functions
7. Query functions

Control functions enable the programmer to ***communicate*** with the window system, initialize programs, deal with errors that occur during the execution of the program etc.

Example :

```
glutInitDisplayMode();
glutInitWindowSize();
glutInitWindowPosition();
```

1. Primitive functions
2. Attribute functions
3. Viewing functions
4. Transformation functions
5. Input functions
6. Control functions
7. Query functions

Query functions enable the programmer to access information such as ***camera parameters***, contents of the frame buffer etc which the programmer can design device independent programs.

Example :

glGetBooleanV();

glGetIntegerV();

OpenGL Interface



The OpenGL Interface

(June 2012)

The three main libraries of OpenGL Interface

1. **Graphics Library (GL)**
2. **Graphics Library Utility (GLU)**
3. **GL Utility Toolkit (GLUT)**

The OpenGL Interface

1. Graphics Library (GL):

- Supports creation of a small set of primitives.
- OpenGL function names begin with the letters **gl.**
- They are stored in a library referred to as *GL (OpenGL in windows).*

The OpenGL Interface

2. Graphics Utility Library (GLU)

- Supports creation of a large set of primitives.
- This library uses only GL functions.
- But it contains code for common objects, such as spheres.
- This library is available in all OpenGL implementations. Functions in the GLU library begin with **glu.**

3. GL Utility Toolkit (GLUT)

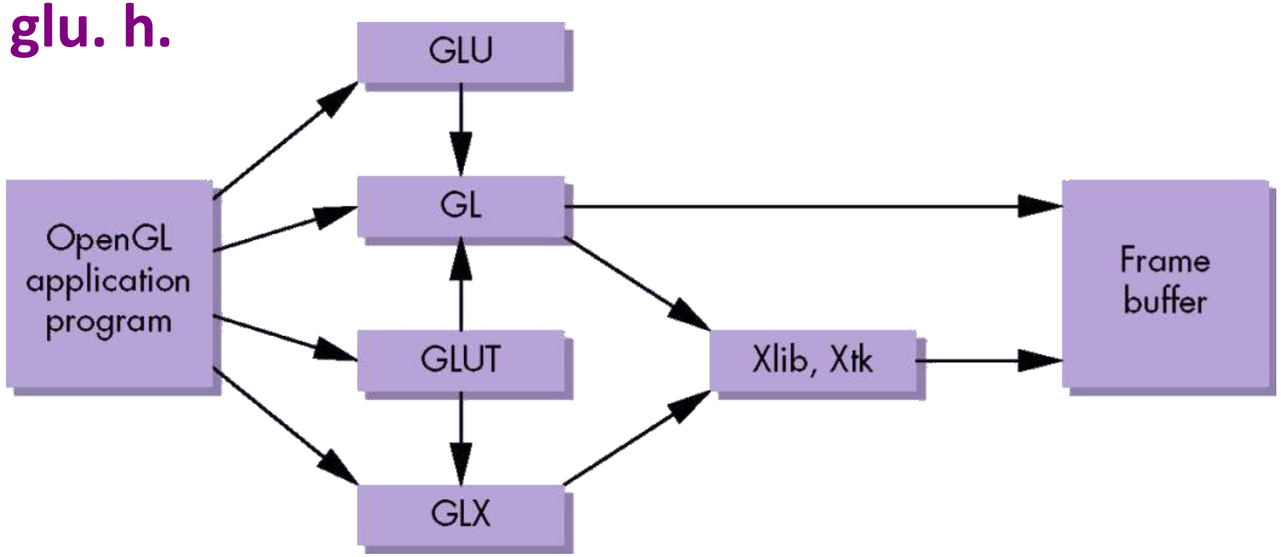
- To interface with the window system and to get input from external devices into the program, one more library is needed.
  - X Windows System - GLX
  - Windows – wgl
  - Macintosh – agl
- But instead of using a diff library every time, we use readily available GLUT – provides min functionality expected in any modern windowing system.

RV College of Engineering

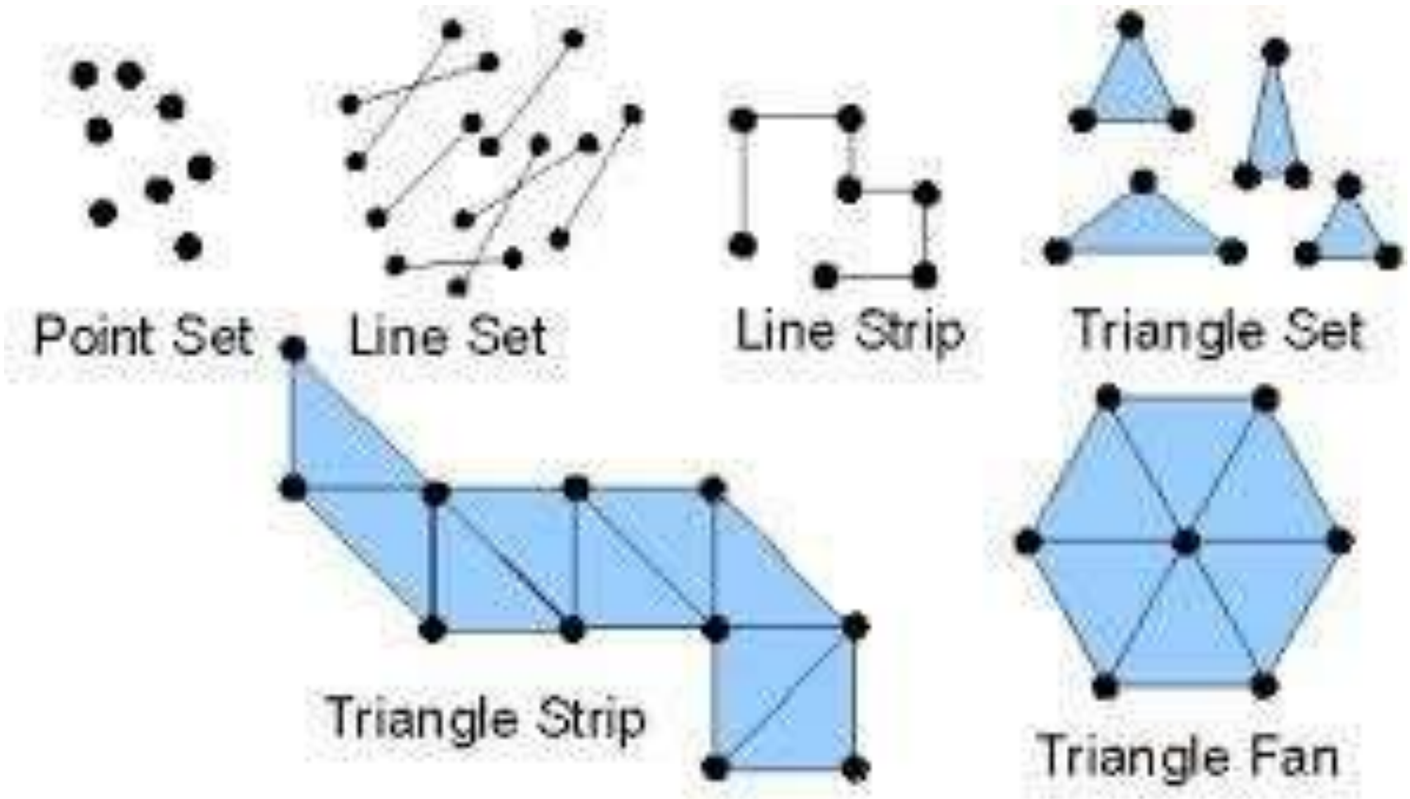
Go, change the world

The OpenGL Interface

- Figure below shows the organization of the libraries for an **X Window system** environment.
- Other libraries are called from the OpenGL libraries, but that the application program does not need to refer to these libraries directly.
- In most implementations, one of the include lines **#include <GL/glut.h>** or **#include <glut.h>** is sufficient to read in **glut.h, gl.h**, and **glu. h.**



OpenGL Primitives & Attributes



RV College of Engineering

OpenGL Primitives & Attributes

Go, change the world

(Jan 2010, Dec 2010)

OpenGL supports the following primitives

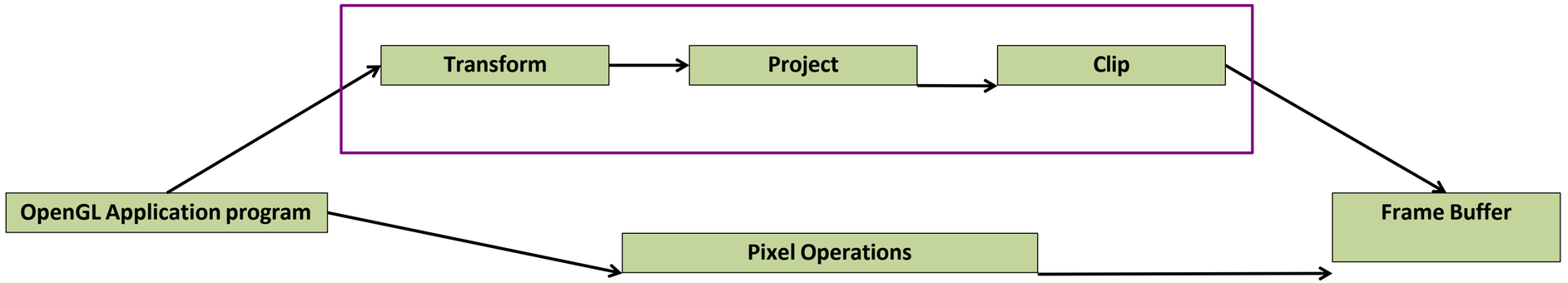
- **Geometric primitives**

  Points, line segments, polygons, curves, surfaces

- **Image OR raster primitives**

  Array of pixels

Geometric pipeline



RV College of Engineering

Go, change the world

Geometric primitives

Geometric primitives syntax :

```
glBegin(type);
    glVertex*(...);
    ..........
    glVertex*(...);
glEnd ( );
```

type specifies how openGL assembles the vertices to define geometric objects.

RV College of Engineering

Go, change the world

Geometric primitives

The geometric primitives and their type specification as follows

1. **Points : (GL_POINTS)**
2. **Line segment : (GL_LINES)**
3. **Polylines : (GL_LINE_STRIP, GL_LINE_LOOP)**

RV College of Engineering

Go, change the world

Programming 2-D Applications

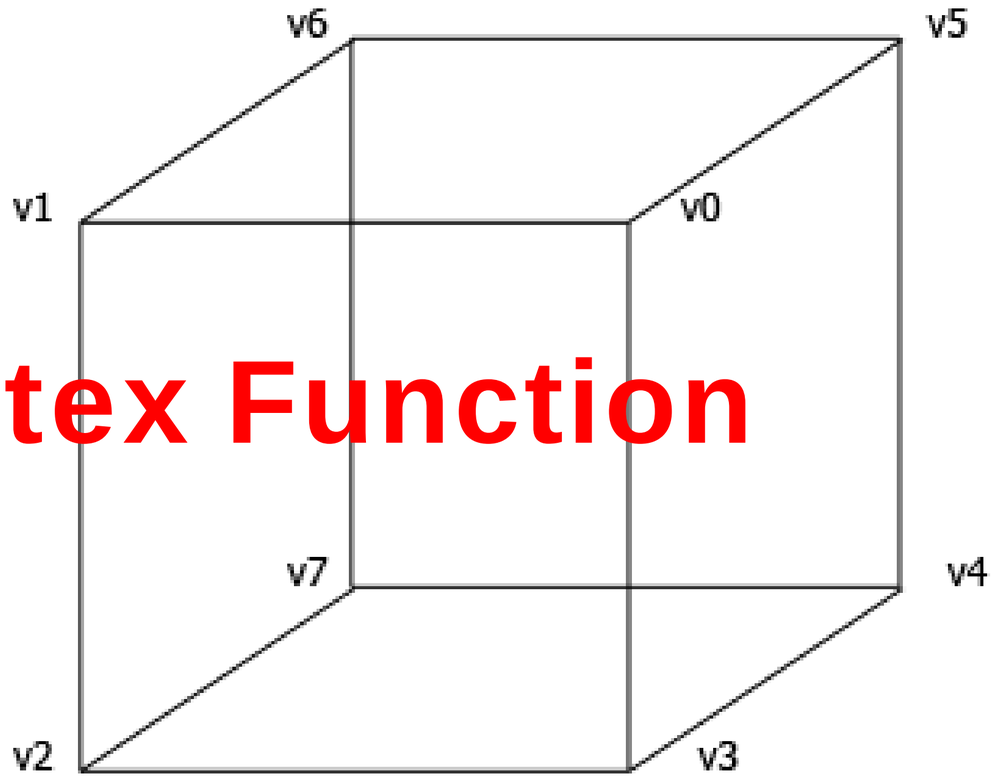
- In OpenGL, internal representation for both 2-D and 3-D points is same. Hence a 3-D point is represented by a triplet:

$$P = \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$

- ***Vertex (* rather than *point):***
  - A vertex is a location in space (in Computer Graphics a 2-D, 3-D, 4-D spaces are used).
  - Vertices are used to define the atomic geometric objects that are recognized by graphics system.

Programming 2-D Applications

- The simplest geometric object is a point in space, which is specified by a single vertex.
- Two vertices define a line segment.
- Three vertices can determine either a triangle or a circle.
- Four vertices determine a quadrilateral, and so on.



Vertex Function

RV College of Engineering

Go, change the world

Vertex Function

- A vertex is a position in space.
- Space can be 2-D or 3-D.
- Vertex is used to define geometric primitives.

Example

Two vertices define a line segment. In OpenGL it is represented as **glVertex*()** where * can be nt OR ntv

n no. of dimensions (2, 3 , 4)

t data types (int, float, double)

v specifies that variables are specified through a pointer to an array.

Basic OpenGL data types include

Glfloat, Glint, GLdouble

RV College of Engineering

Go, change the world

Vertex Function

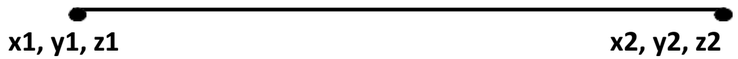
Vertex in 2D can be represented as
glVertex2i(GLint xi, GLint yi)

In OpenGL, primitives are defined between ***glBegin*** and ***glEnd***

glBegin – The argument specifies the geometric type of vertices.

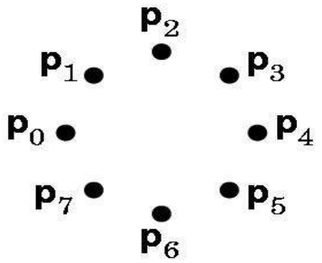
Example :

```
glBegin(GL_LINES);
   glVertex3f(x1, y1, z1);
   glVertex3f(x2, y2, z2);
glEnd();
```

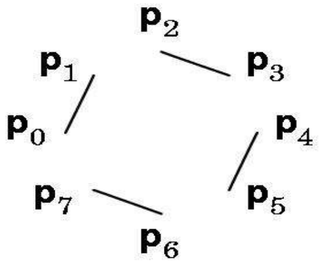


RV College of Engineering

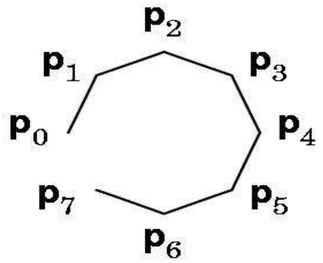
Go, change the world



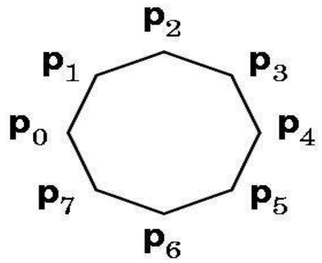
GL_POINTS



GL_LINES



GL_LINE_STRIP



GL_LINE_LOOP

GL_POINTS : Each vertex is displayed at a size of at least one pixel.

GL_LINES : Successive pairs of vertices would be connected as a line.

GL_LINE_STRIP : Connects the successive vertices using line segments. However the final vertex would not be connected to the initial vertex.

GL_LINE_LOOP : Connects the successive vertices using line segments to form a closed path.

OpenGL Point Attributes

If there is a Point, then it will contain 2 Attributes:

1. Size 2. Color

The size of a point can be increased or decreased using glPointSize().

Point	Attribute	OpenGL Function
	Size	glPointSize
	Color	glColor3f

```
glPointSize(int size);
```
void

```
glColor3f(r, g, b);
```
Sets the point color

Point



Attribute

Size

Color

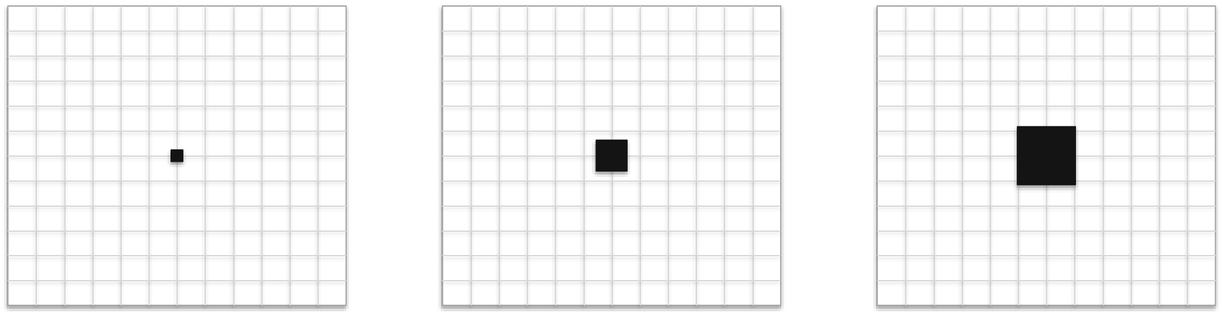
OpenGL Functions

glPointSize

glColor3f

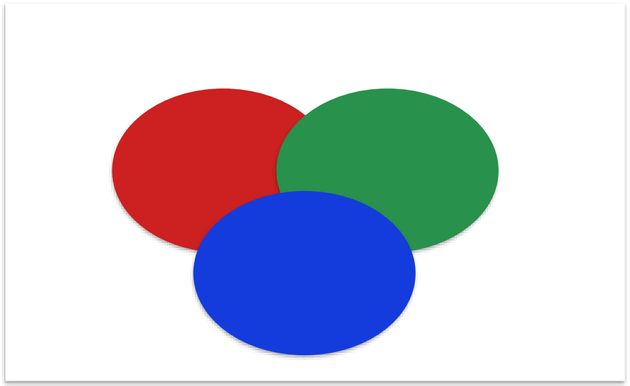
By Default the Point Size is 1 (One Pixel)

Size is always Square Block of Pixel[s]



void glPointSize(int size)

Point	Attribute	OpenGL Functions
	Size	glPointSize
	Color	glColor3f



void glColor3f(float R,float G, float B)

glColor3f(1,0,0);

Line	Attribute	OpenGL Functions
	Width Style Color	

Line	Attribute	OpenGL Functions
	Width	glLineWidth
	Style	

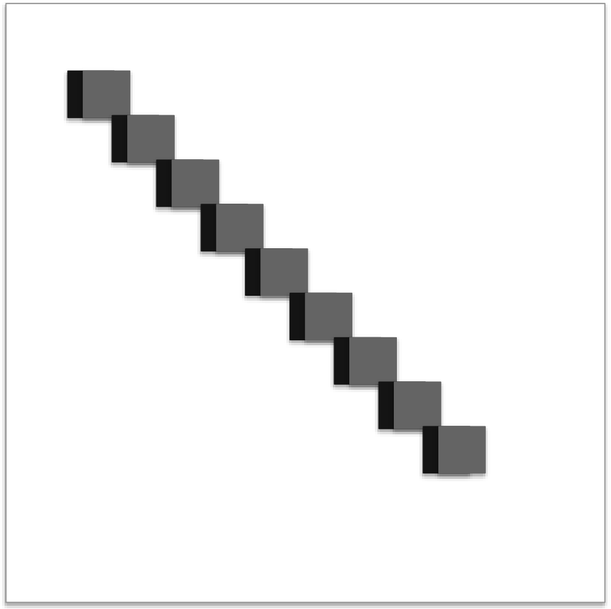
void glLineWidth(width)

Parameter width can be int/float

Default Width is 1

Pixel Spans for Thickness/Width

Vertical Pixel Spans or Horizontal Pixel Spans

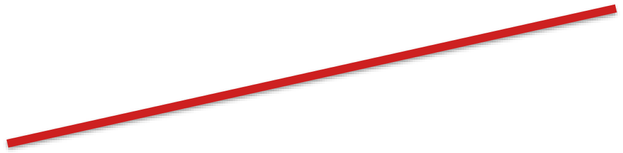


Line

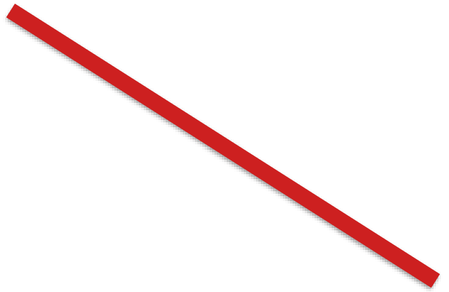
Attribute
Width
Style

OpenGL Functions
glLineWidth

```
glLineWidth(5);
glBegin(GL_LINES);
    glVertex2f(-0.5,0.5);
    glVertex2f(0.25,0.75);
glEnd();
```



```
glLineWidth(10);
```



Line	Attribute	OpenGL Functions
	Style	glLineStipple

glLineStipple (repeatFactor, pattern)

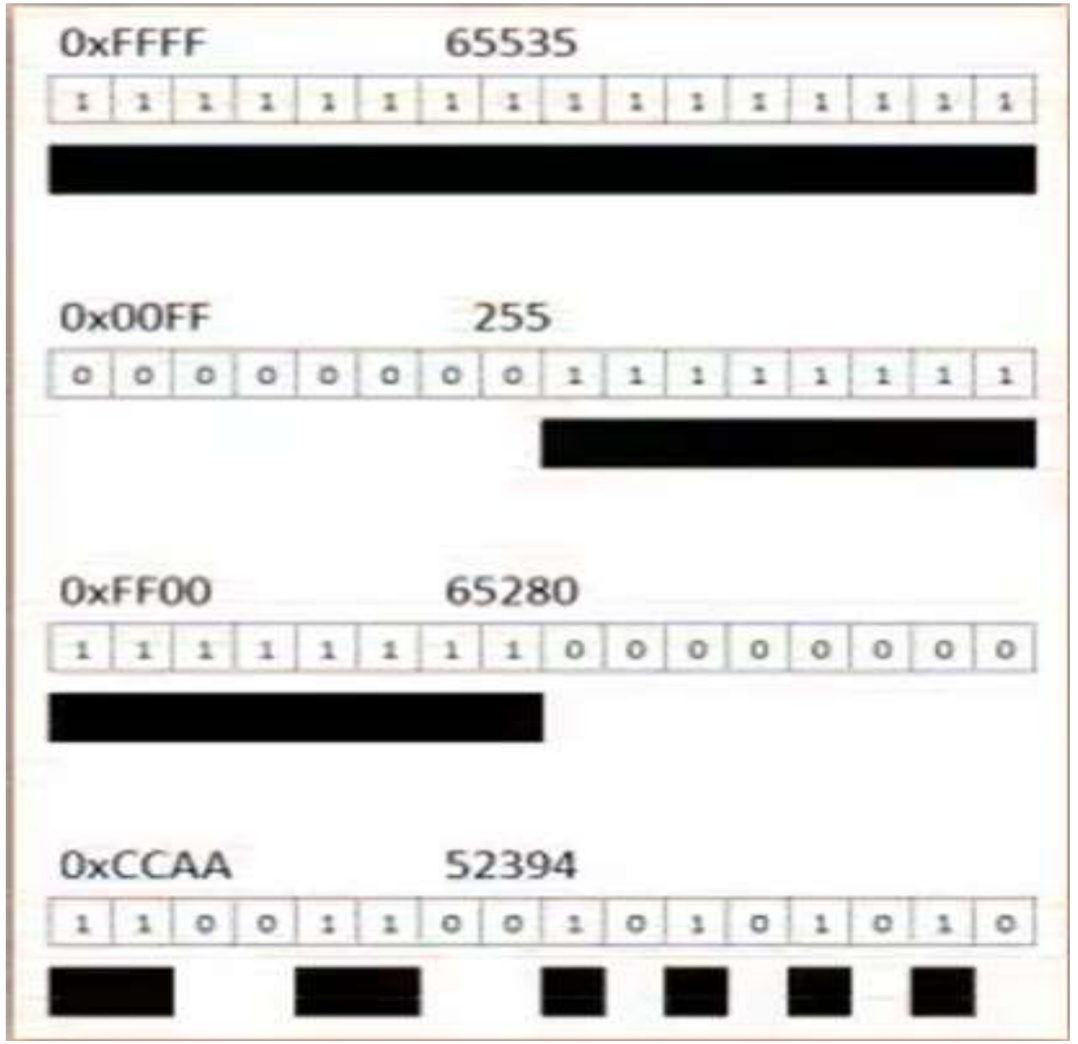
16-bit Integer to describe the Line Style

Which Decides the Pattern

Starting with the Low-Order Bits

Repeat of Pattern

Applied continuously in case of Polyline



Line

Attribute

Style

OpenGL Functions

glLineStipple

glLineStipple (repeatFactor, pattern)

```
glLineWidth(5);
glLineStipple (1, 0xFFFF);
```



```
glLineStipple (1,255);
```



```
glLineStipple (1,0xFF00);
```



```
glLineStipple (1,0xCCAA);
```

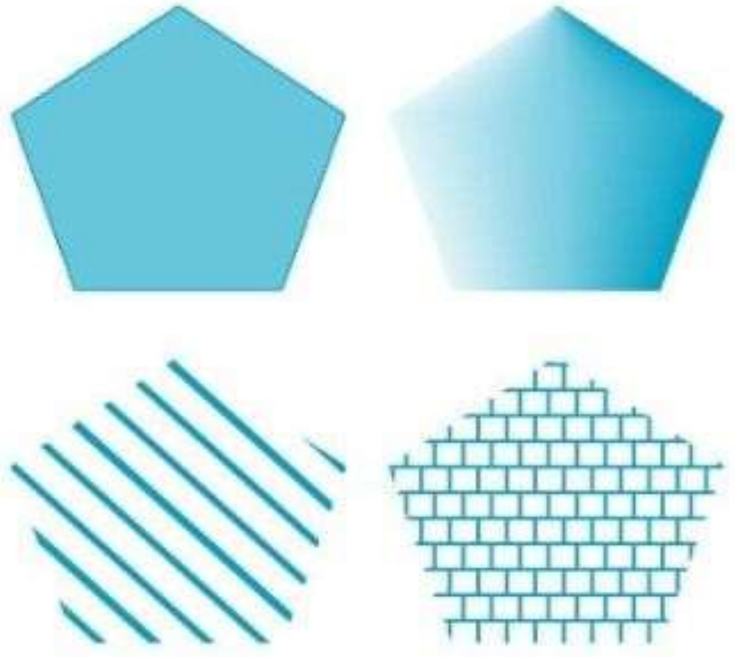
Type	Attribute	OpenGL Functions
Point	Color	glColor3f
Line	Size	glPointSize
Curve	Width	glLineWidth
	Style	glLineStipple
		glEnable (GL_LINE_STIPPLE)
		glDisable (GL_LINE_STIPPLE)

RV College of Engineering

Go, change the world

Polygon Basics:Ways of displaying polygons

- Only its edges be displayed.
- Its interior be filled with a solid color, or a pattern.
- Either display or not display the edges.
- Outer edges of a polygon can be defined easily by an ordered list of vertices.
- But if the interior is not well defined, then the polygon may be rendered incorrectly.

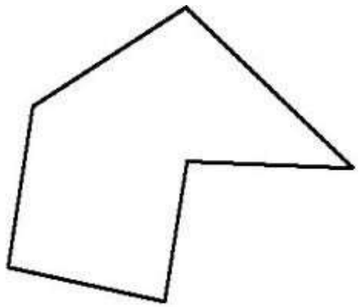


Properties of a polygon

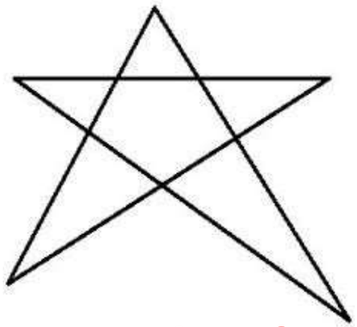
Three properties will ensure that a polygon will be displayed correctly. It must be **simple, convex, and flat.**

/. Simple:

- A 2-D polygon in which **no pair of edges cross each other.**
- They will have well-defined interiors.



Simple

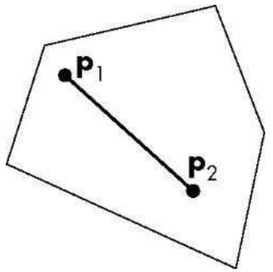


Non simple

Properties of a polygon

s. Convex

- A convex polygon is a multi-angled shape with straight sides in which all of the angles are **< 180^0**.
- A **pentagon** is an example of a convex polygon.
- A line drawn through a convex polygon will intersect the sides of the polygon **exactly twice.**
- All triangles and regular polygons such as squares, parallelograms, trapezoids, and rectangles are convex because each of their interior angles is less than 180^0.

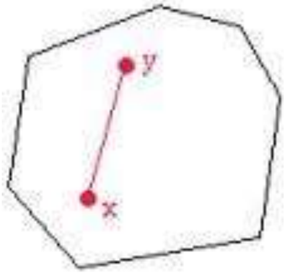


RV College of Engineering

Go, change the world

Properties of a polygon

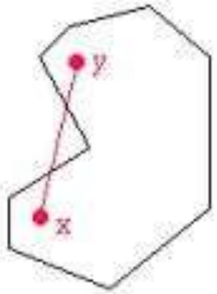
s. Convex



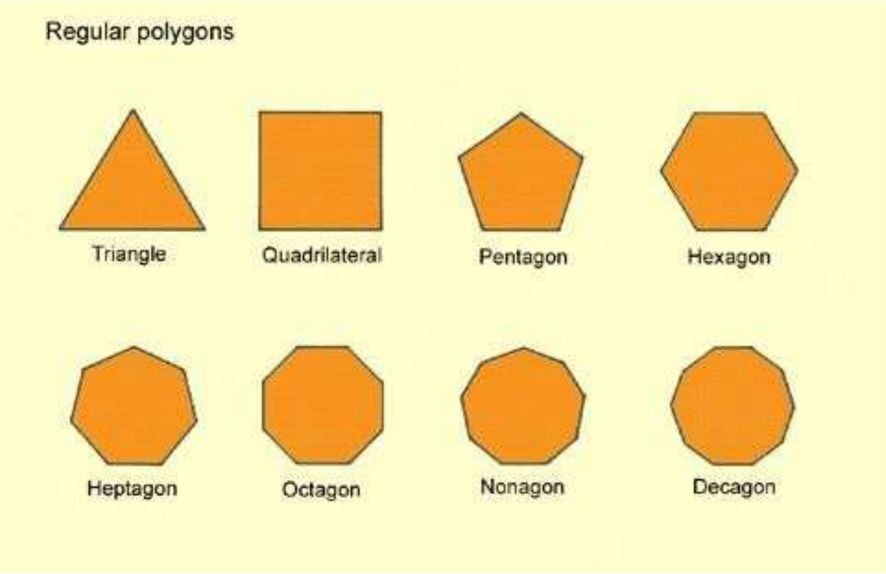
A convex polygon







A non–convex polygon

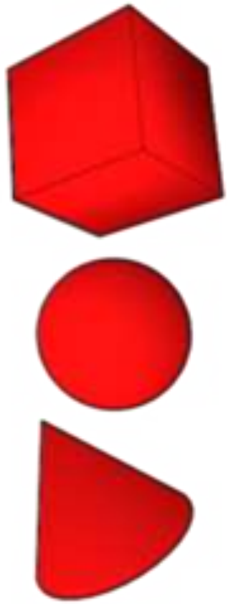


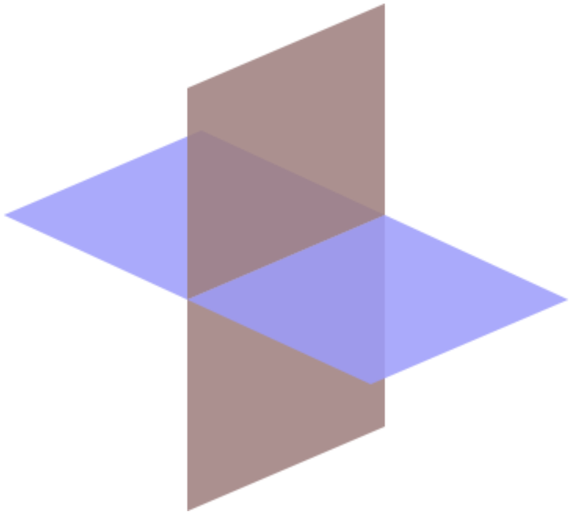
OpenGL

Properties of a polygon

3. **Flat**

- Flat defines that the vertices lie in the **same plane.** (A plane is a perfectly flat surface extending in all directions).
- In 3-D, **all vertices** that define polygon **need to lie in the same plane**.





RV College of Engineering

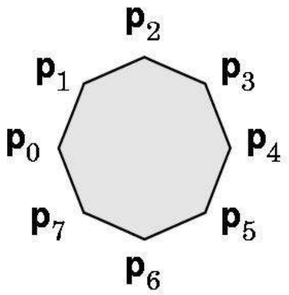
Go, change the world

Polygon Types in OpenGL

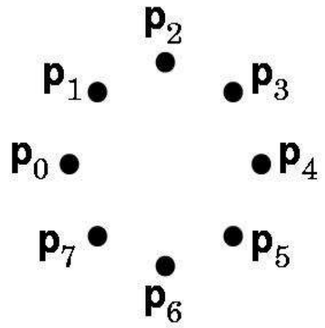
GL_POLYGON : Connects the successive vertices using line segments to form a closed path. The interior is filled according to the state of the relevant attributes.

GL_QUADS : Special case of polygon where successive group of 4 vertices are interpreted as quadrilaterals.

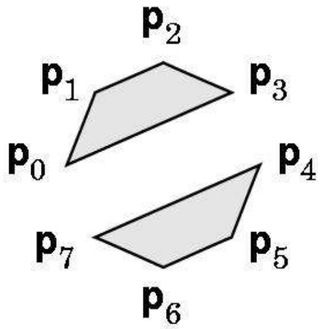
GL_TRIANGLES : Special case of polygon where successive group of three vertices would be interpreted as a triangle.



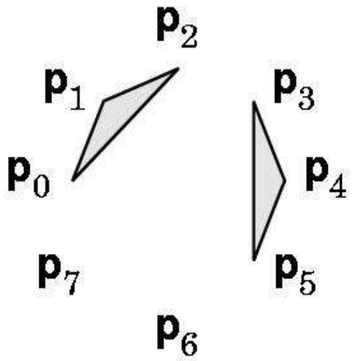
GL_POLYGON



GL_POINTS



GL_QUADS



GL_TRIANGLES

OpenGL

OpenGL Primitive Strips

GL_TRIANGLE_STRIP

GL_QUAD_STRIP

GL_TRIANGLE_FAN

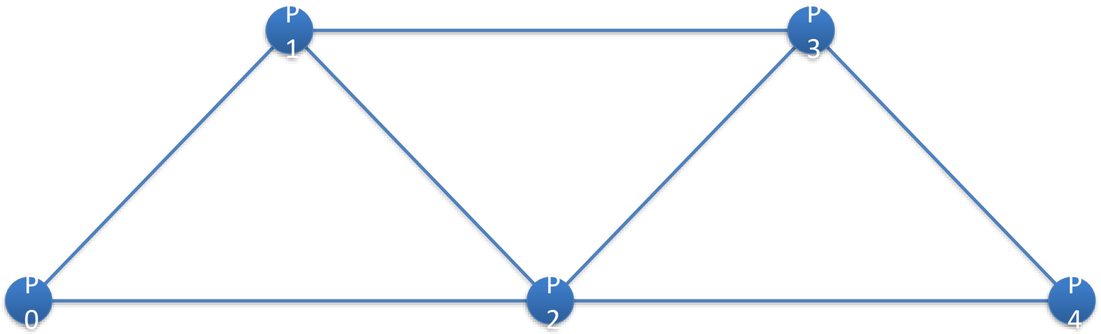
1. GL_TRIANGLE_STRIP — "Make triangles continuously"

Basic idea

In a triangle strip, every new point creates a new triangle using the previous two points.

Suppose we give OpenGL these points:

P0, P1, P2, P3, P4...

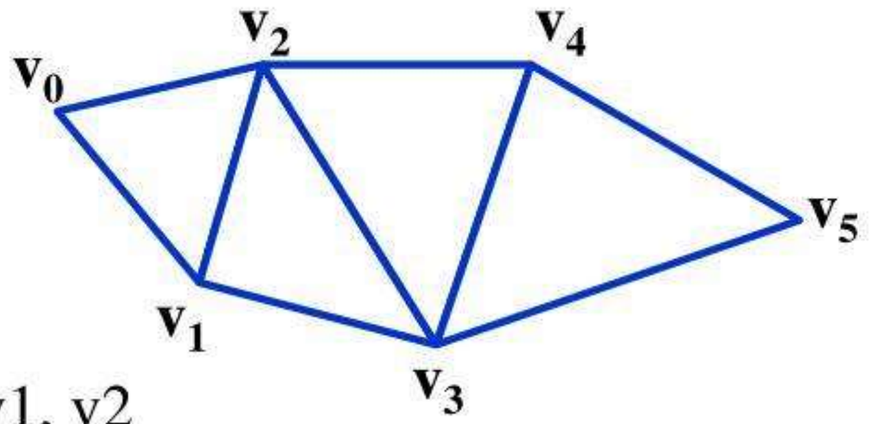




OpenGL: Triangle Strips

- An OpenGL *triangle strip* primitive reduces this redundancy by sharing vertices:

```
glBegin(GL_TRIANGLE_STRIP);
glVertex3fv(v0);
glVertex3fv(v1);
glVertex3fv(v2);
glVertex3fv(v3);
glVertex3fv(v4);
glVertex3fv(v5);
glEnd();
```



  - triangle 0 is v0, v1, v2
  - triangle 1 is v2, v1, v3 (*why not v1, v2, v3?*)
  - triangle 2 is v2, v3, v4
  - triangle 3 is v4, v3, v5 (again, **not** v3, v4, v5)

OpenGL – Triangle Strip (Why not V1, V2, V3?)

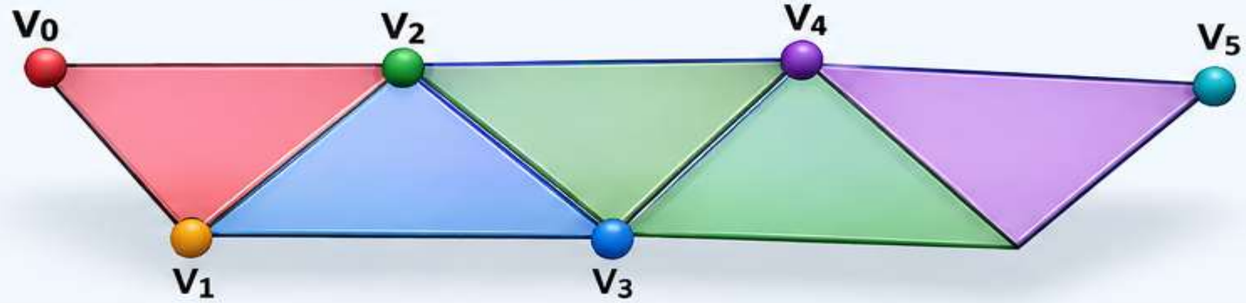
1. The Vertex Order

OpenGL receives the vertices in this order:

V0 → V1 → V2 → V3 → V4 → V5

Each new vertex creates a triangle with the previous two vertices.

2. 3D View of the Triangle Strip



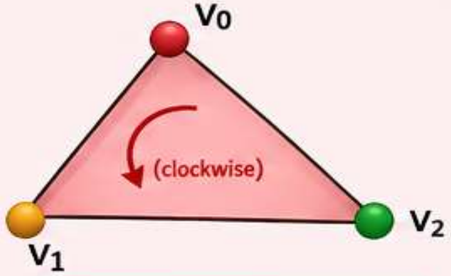
The strip forms connected triangles.

Notice the shared edge between triangles.

3. Triangles Formed (with direction)

Triangle 0 (First Triangle)

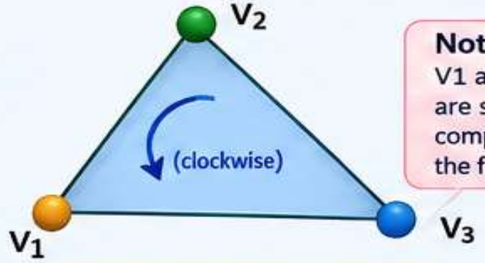
Vertices: V0, V1, V2



Order: V0 → V1 → V2

Triangle 1 (Second Triangle)

Vertices: V2, V1, V3 (not V1, V2, V3)

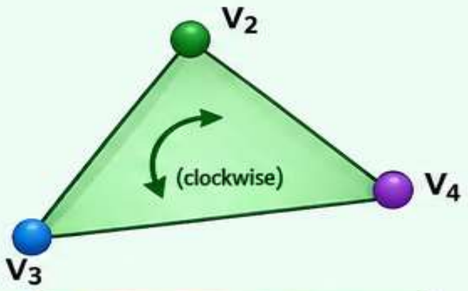


Notice: V1 and V2 are swapped compared to the first triangle.

Order: V2 → V1 → V3

Triangle 2 (Third Triangle)

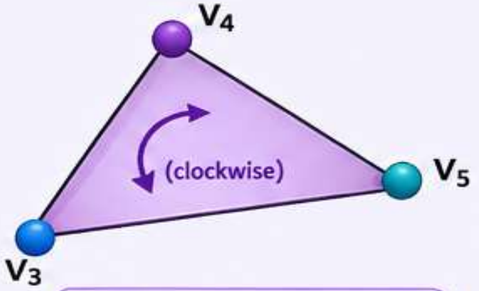
Vertices: V2, V3, V4



Order: V2 → V3 → V4

Triangle 3 (Fourth Triangle)

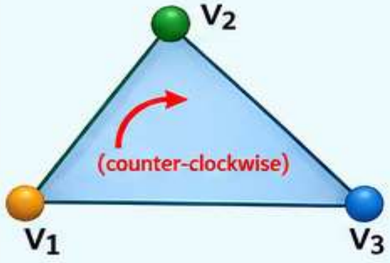
Vertices: V4, V3, V5



Order: V4 → V3 → V5

4. What if we used V1, V2, V3 for Triangle 1?

Let's check the direction:



✗ V1 → V2 → V3 gives the opposite winding direction compared to the first triangle.

This breaks the consistent orientation of the strip.

5. Why OpenGL does this?

OpenGL alternates the order of the first two vertices for every next triangle so that all triangles have the same winding direction (orientation).

✓ This helps with:

- Correct front-face / back-face detection
- Face culling (removing hidden triangles)

Quick Summary

Triangle No.	Vertices (Order)	Direction
0	V0, V1, V2	Clockwise ↻
1	V2, V1, V3	Clockwise ↻
2	V2, V3, V4	Clockwise ↻
3	V4, V3, V5	Clockwise ↻

Remember:
Every new vertex makes a triangle with the previous two, but OpenGL alternates the order to keep the direction consistent.

GL_TRIANGLE_STRIP — How triangles are formed

OpenGL connects them like this:

- V0, V1, V2 → Triangle 1
- V1, V2, V3 → Triangle 2
- V2, V3, V4 → Triangle 3
- V3, V4, V5 → Triangle 4

So, after the first 3 points, each additional point creates one more triangle.

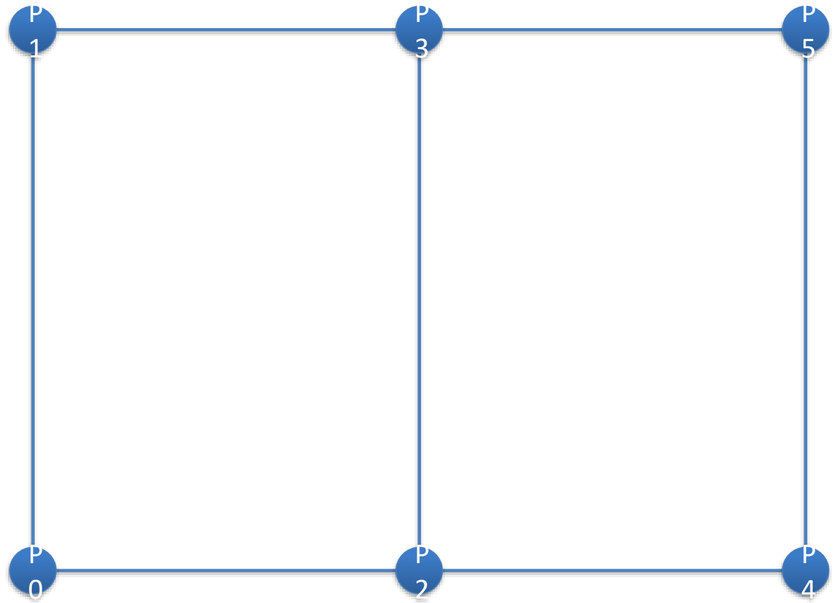
2. GL_QUAD_STRIP — "Make quadrilaterals continuously"

A quad is a four-sided shape.

For example:

P1 -------- P3
 | |
 | |
P0 -------- P2

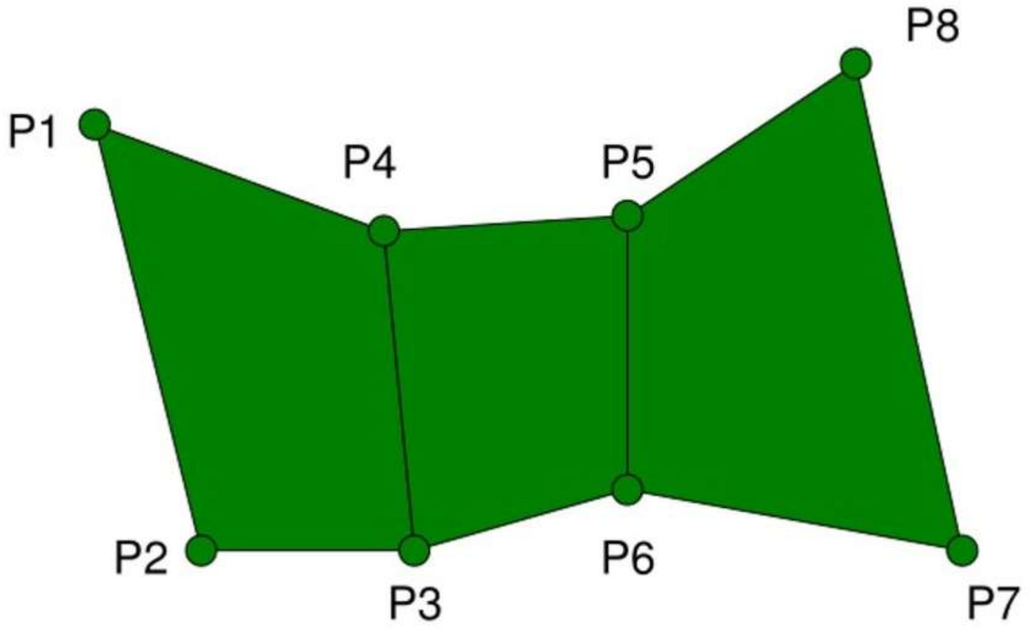
These four points create one quadrilateral.



OpenGL Drawing Functions

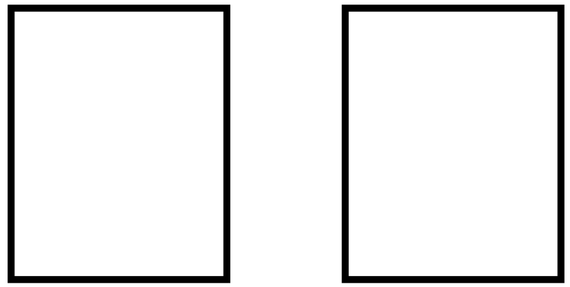
```
glBegin (GL_QUADS_STRIP);
    glVertex2iv (p1);
    glVertex2iv (p2);
    glVertex2iv (p4);
    glVertex2iv (p3);
    glVertex2iv (p5);
    glVertex2iv (p6);
    glVertex2iv (p8);
    glVertex2iv (p7);

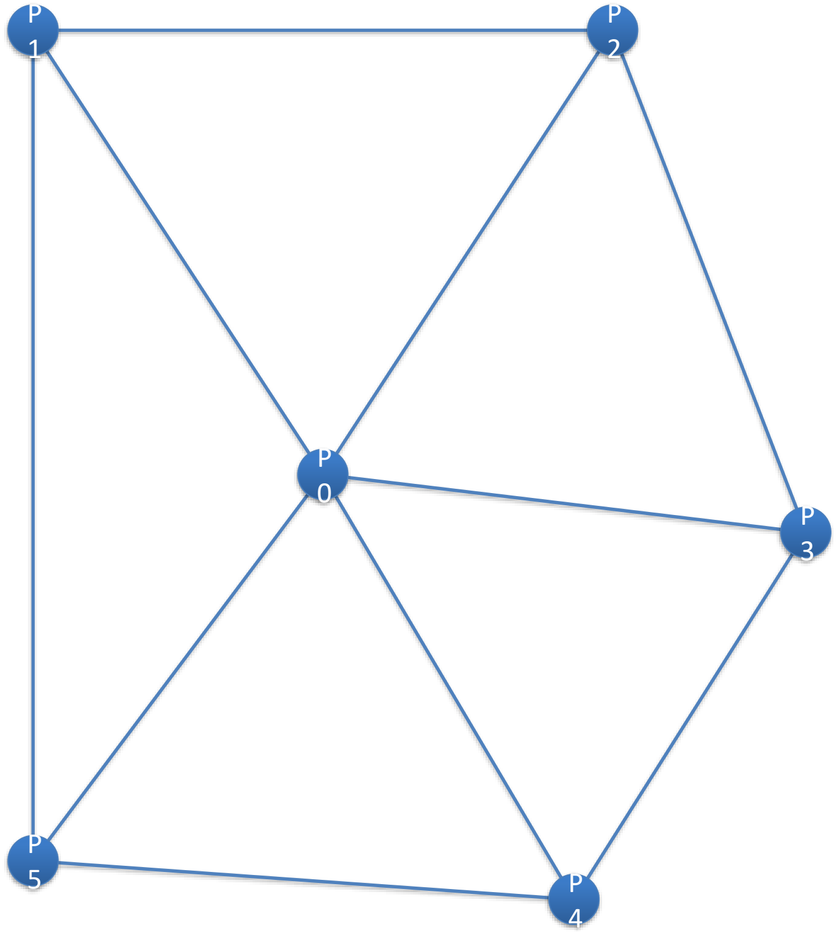
glEnd();
```



3. GL_TRIANGLE_FAN — "One fixed point + triangles around it"

This one is very easy if you imagine a fan





There is one fixed point in the center.

Let's call it P0.

Then we give:

P1, P2, P3, P4, P5...

GL_TRIANGLE_FAN — How triangles are formed

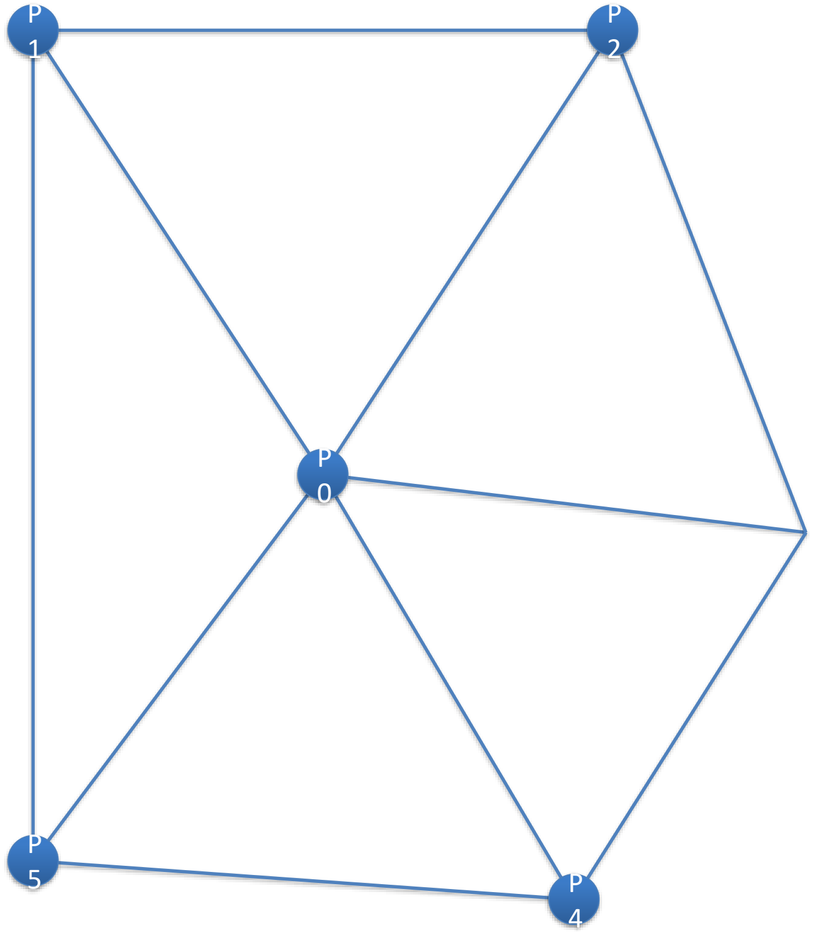
The important point is:

P0 always remains fixed.

Triangles are:

- P0, P1, P2 → Triangle 1
- P0, P2, P3 → Triangle 2
- P0, P3, P4 → Triangle 3
- P0, P4, P5 → Triangle 4

So every new point is connected back to P0.



⭐ One-line memory trick

TRIANGLE STRIP → Previous 2 points + New point

QUAD STRIP → Previous pair + New pair

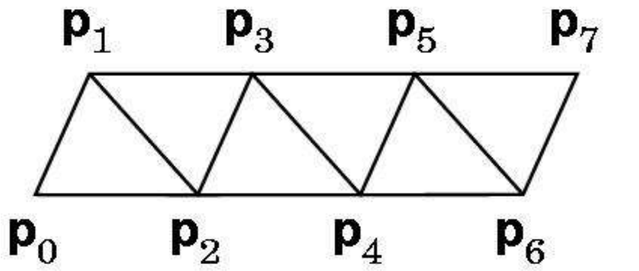
TRIANGLE FAN → Fixed center + New outer point

RV College of Engineering

Go, change the world

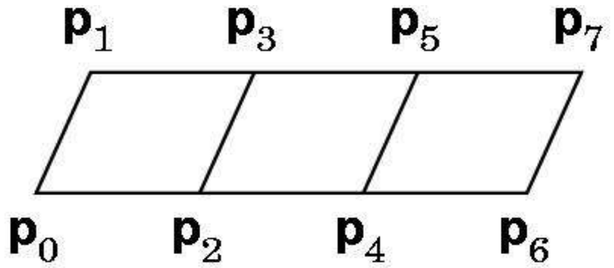
Polygon Types in OpenGL

GL_TRIANGLE_STRIP : Each additional vertex is combined with the previous two vertices to define a new triangle.



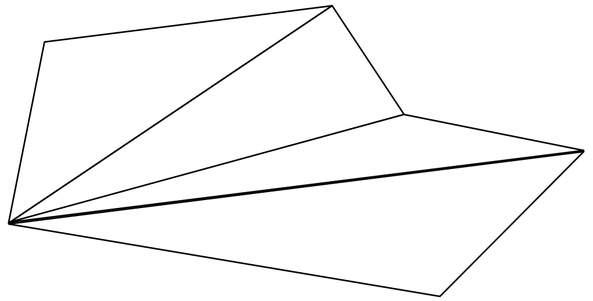
GL_TRIANGLE_STRIP

GL_QUAD_STRIP : Two new vertices are combined with the previous two vertices to design a new quadrilateral.



GL_QUAD_STRIP

GL_TRIANGLE_FAN : Based on one fixed point. The next two points determine the first triangle. The subsequent triangles are formed from one new point, the previous point and the first (fixed) point.

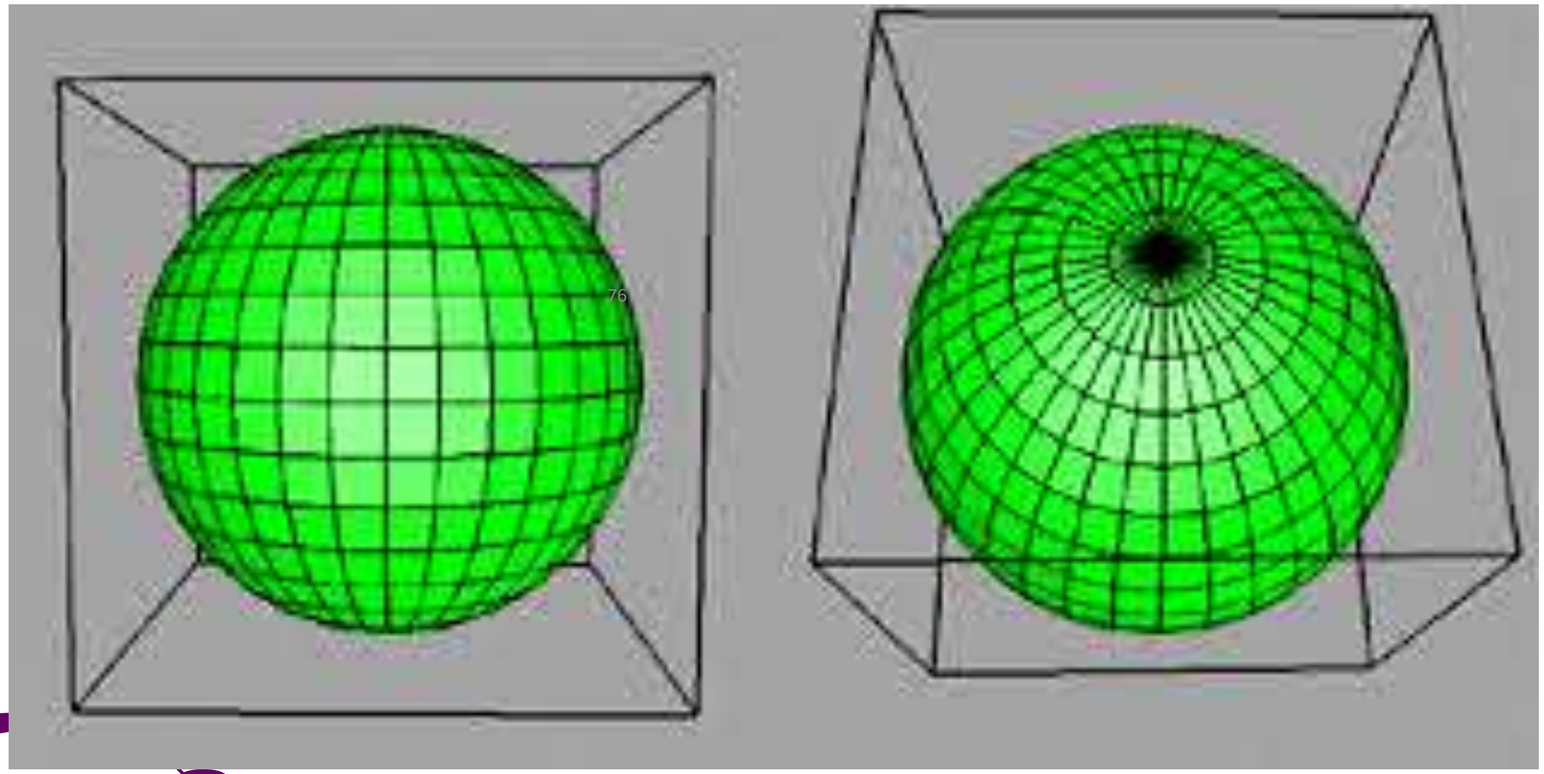


GL_TRIANGLE_FAN

Curved Primitives

- Curved primitives such as sphere, oval, parabola can be created using ***tessellation.***
- ***Tessellation*** is a process of creating curved primitives by using a mesh of polygon of n sides.
- ***Example*** : Approximating a Sphere

Approximating a Sphere



Approximating a Sphere

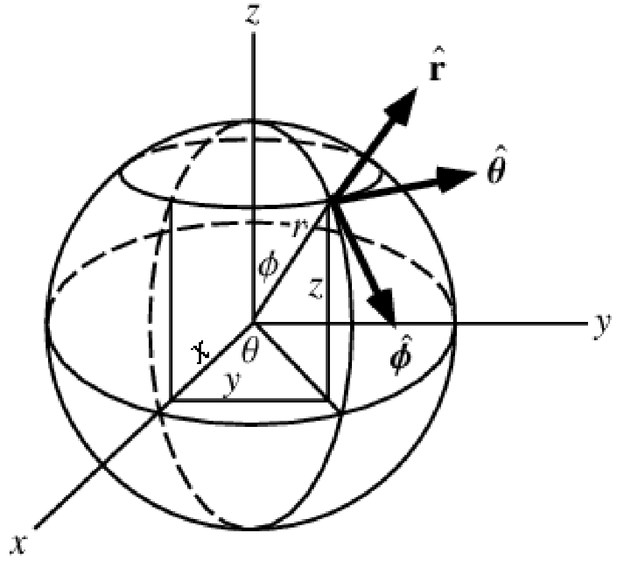


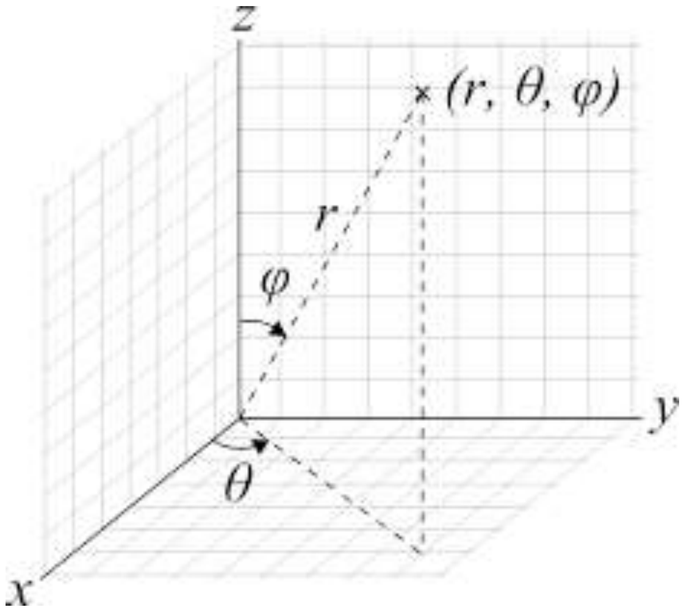
Approximating a Sphere

- Many curved surfaces can be approximated using fans and strips.
- E.g. To approximate to a sphere, **a set of polygons defined by lines of longitude and latitude** as shown can be used.
- Either quad strips or triangle strips can be used for the purpose.
- Consider a unit sphere (r=1). It can be described by the following three equations: $x(\theta, \emptyset) = \sin\theta \cos\emptyset$

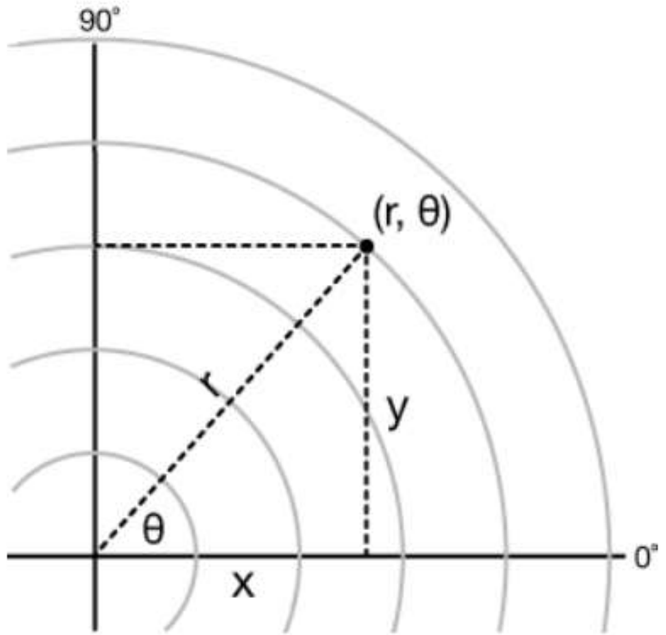
$y(\theta, \emptyset) = \cos\theta \cos\emptyset$

$z(\theta, \emptyset) = \sin\emptyset$



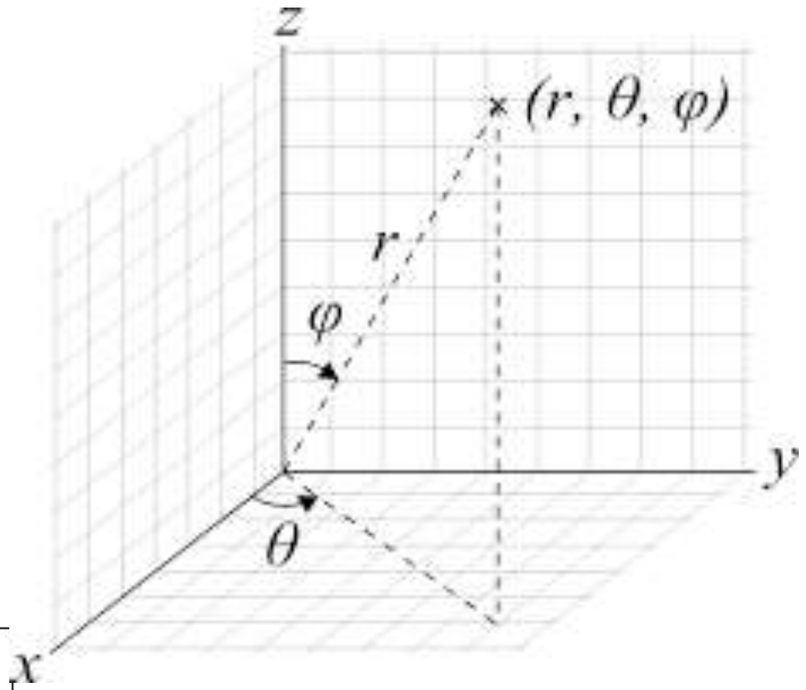


Approximating a Sphere



$$x = r\cos(\theta) \qquad y = r\sin(\theta)$$

$$r^2 = x^2 + y^2 \qquad \theta = \tan^{-1}\left(\frac{y}{x}\right)$$

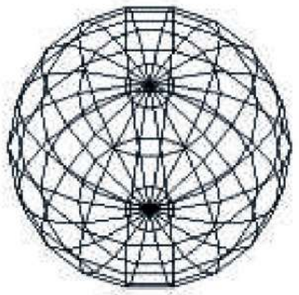


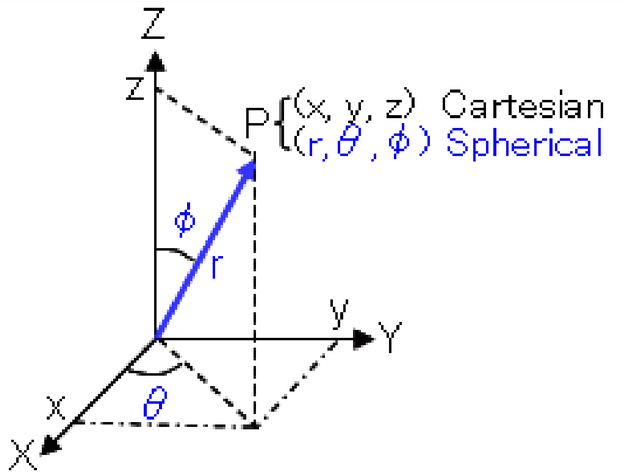
x(θ, Ø)= sinθ cosØ
y(θ, Ø)= cosθ cosØ
z(θ, Ø) = sinØ

OpenG

Approximating a Sphere

$x(\theta, \emptyset) = \sin\theta \cos\emptyset$
$y(\theta, \emptyset) = \cos\theta \cos\emptyset$
$z(\theta, \emptyset) = \sin\emptyset$





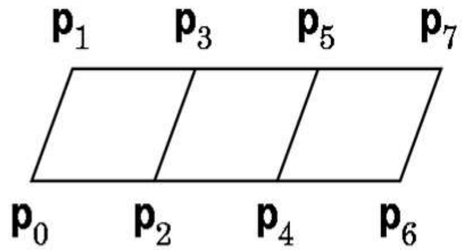
- If we **fix Ø** and draw curves, as we change θ, we get circles of **constant latitude.**
- If we **fix θ** and draw curves, as we change Ø, we get circles of **constant longitude.**
- By generating points at fixed increments of θ and Ø, we can define quadrilaterals.
- Ø must be ranged between -80 to +80.
- θ must be ranged between -180 and 180.

Approximating a Sphere

- Degrees must be converted to radians for the standard trigonometric functions.
- **Code :**

```
float c=3.142/180; // converting to radians
// to get longitudes
for(phi=-80.0; phi<=80.0; phi+=20.0)
{
        phir=c*phi;  // φ in radians – for the 1st point
        phir20=c*(phi+20);  // for the 2nd point
```

Approximating a Sphere




```
glBegin(GL_QUAD_STRIP);
// to get latitudes
for (theta=-180.0; theta<=180.0; theta+=20.0)
{
      thetar=c*theta;
      x=sin(thetar)*cos(phir);
      y=cos(thetar)*cos(phir);
      z=sin(phir);
      glVertex3d(x,y,z); // 1st point
```

Approximating a Sphere

```
            //phir20=c*(phi+20);
            x=sin(thetar)*cos(phir20);
            y=cos(thetar)*cos(phir20);
            z=sin(phir20);
            glVertex3d(x,y,z); // 2nd point
        }
        glEnd();
}
```

Approximating a Sphere



```
//1st pole – use triangle fans
glBegin(GL_TRIANGLE_FAN);
glVertex3d(0.0,0.0,1.0);   // top pole
c80=c*80.0;
z=sin(c80);
for(theta=-180.0;theta<=180.0;theta+=20.0)
{     thetar=c*theta;
      x=sin(thetar)*cos(c80);
      y=cos(thetar)*cos(c80);
      glVertex3d(x,y,z);
}
glEnd();
```

Approximating a Sphere

```
//2nd pole – use triangle fans
glBegin(GL_TRIANGLE_FAN);
glVertex3d(0.0,0.0,-1.0); // bottom pole
z=-sin(c80);
for(theta=-180.0;theta<=180.0;theta+=20.0)
{
      thetar=c*theta;
      x=sin(thetar)*cos(c80);
      y=cos(thetar)*cos(c80);
      glVertex3d(x,y,z);
}
glEnd();
```



TEXT



Text

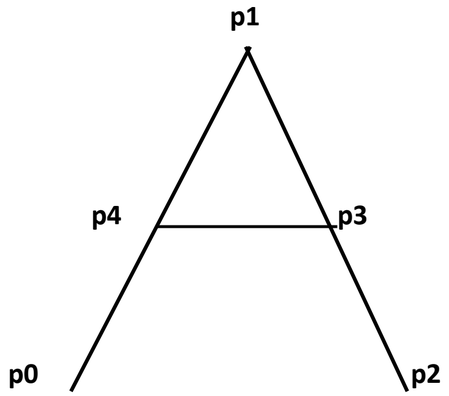
- In computer graphics, text may need to be displayed in a multitude of **fashions** by controlling **type styles, sizes, colors, and other parameters.**
- Two types
  - **Stroke text**
    - **void glutStrokeCharacter(void *font, int character)**
  - **Raster text**
    - **void glutBitmapCharacter(void *font, int character)**

RV College of Engineering

Go, change the world

Stroke Text

- It is constructed **similar to geometric objects.**
- Vertex is used to define the line segment or curves that outline each character.
- The characters defined by closed boundaries can be filled.
- Stroke fonts behave like any 3D object, i.e. the characters can be rotated, scaled, and translated.





Stroke Text

Advantages

- It can be translated, rotated, scaled etc.
- It can be viewed using different views.

Disadvantages

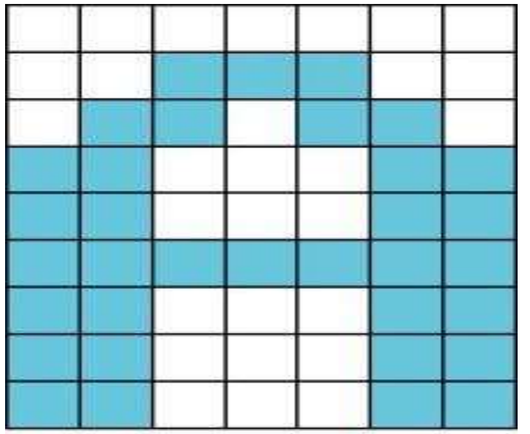
- Consumes lot of memory.
- Consumes lot of processing time.

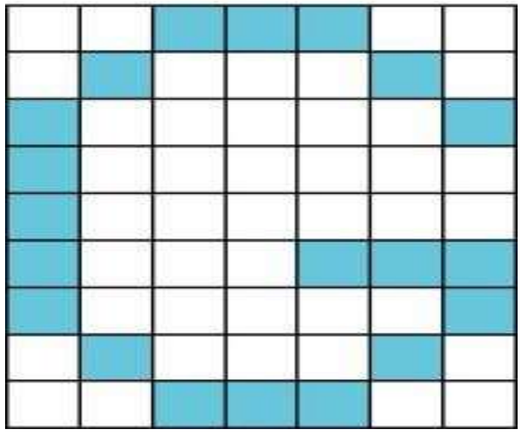
RV College of Engineering

Go, change the world

Raster text

- It is simple and fast.
- Characters are defined as rectangles of bits called **bit blocks**.
- **Each block defines a single character by the pattern of 0 and 1 bits in the block.**
- A raster character can be placed in the frame buffer rapidly by a **bit-block-transfer operation**, which **moves the block of bits using a single instruction.**





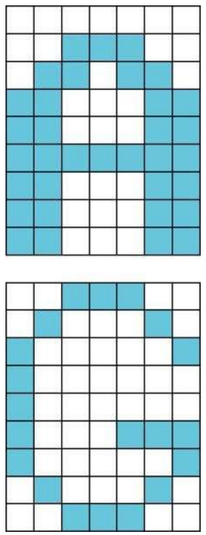
RV College of Engineering

Go, change the world

Raster text

- OpenGL allows the application program to use instructions that allow direct manipulation of the contents of the frame buffer.
- **Size of raster characters** can be **increased only by replicating or duplicating pixels**, a **process that gives larger characters a blocky appearance.**





Advantages

- Can be placed in the frame buffer rapidly by a bit block transfer.
- Simple and fast.

Disadvantages

- Rotation, scaling etc cannot be applied.
- Raster characters often are stored in read-only memory (ROM) in the hardware and hence, a particular font might be of limited portability.

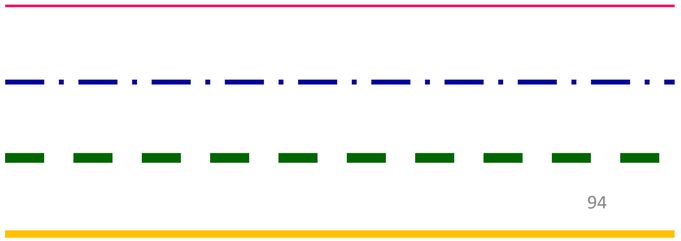
Attributes (July 2011)

- Property of a geometric primitive that refers to its ***color, thickness, pattern, edged, non-edged*** etc.
- Each geometric type has a set of attributes such as
  - A **point** has **color** and **thickness** attributes.
  - A **line** has **color, thickness** and **pattern** attributes.
  - A **polygon** has **color, thickness, pattern, display only edges, display without edges attributes.**
- **Type of the primitive differs from its attribute**.

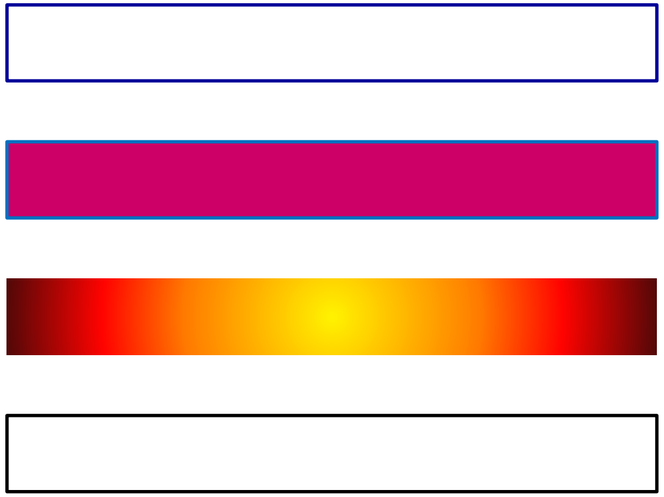
  E.g. ***A red solid line and a green dashed line are the same geometric type, but each is displayed differently.***
- Attribute defines the nature of the geometric primitive and appearance of objects. ***glColor3f(); glColorClear();***

Attributes for lines and polygons

Lines



Polygons



Color



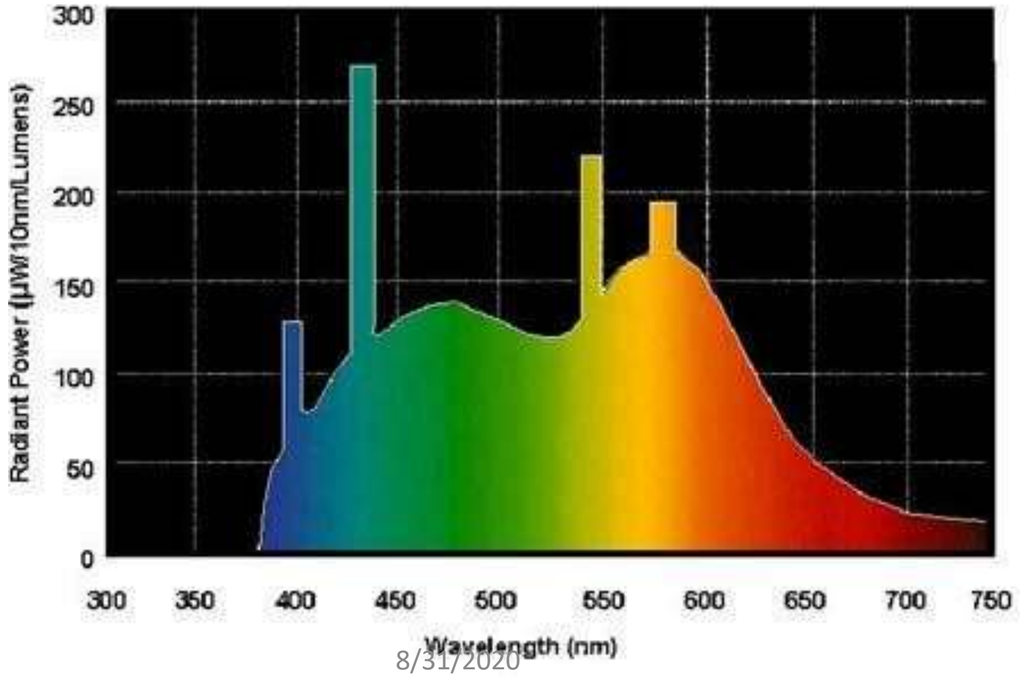
RV College of Engineering

Go, change the world

Color

It is the most important attribute of any geometric primitive.

A visible color can be characterized by function C(X) that occupies wavelengths from about 350 nm to 780 nm.



Color

- The human visual system has 3 types of cones responsible for color vision.
- Hence our brains do not receive the entire distribution for the given color but 3 values – **tristimulus values** – that are responses of the **3 types of cones to the color.**
- This reduction of a color to 3 values leads to the basic tent of three-color theory.

RV College of Engineering

Tristimulus Values

Go, change the world

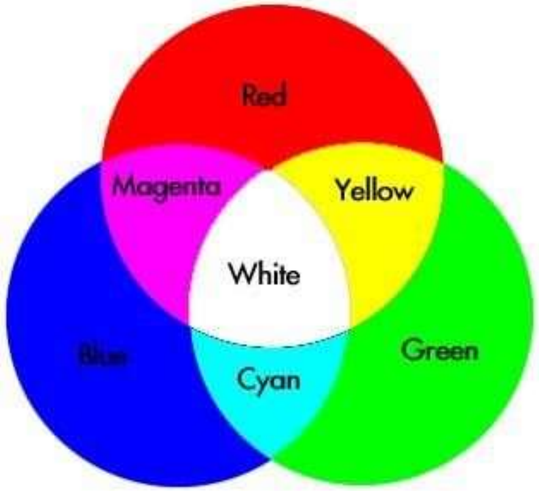
- Any color which can be produced by three primary colors - blue, green, and red and can be written as:

$$C = rR + gG + bB$$

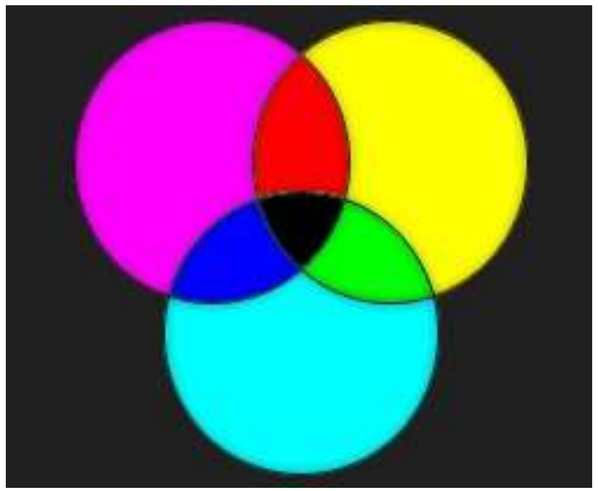
- Where **R,G,B** can be considered to be "unit values" for blue, green, and red.
- r,g,b are the magnitudes or relative intensities of those primaries and are called "tristimulus values".

Color modes

Additive color



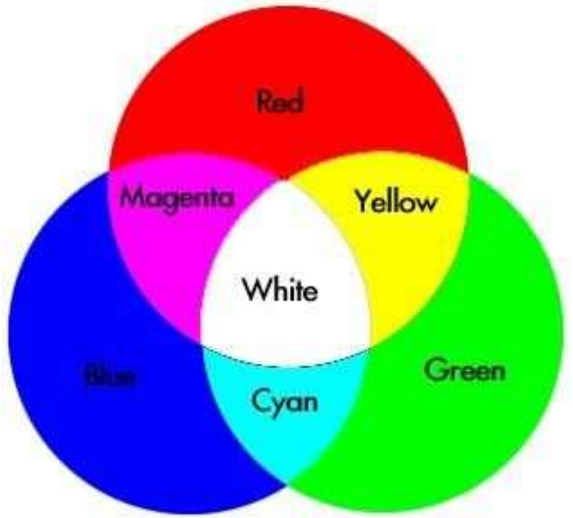
Subtractive color



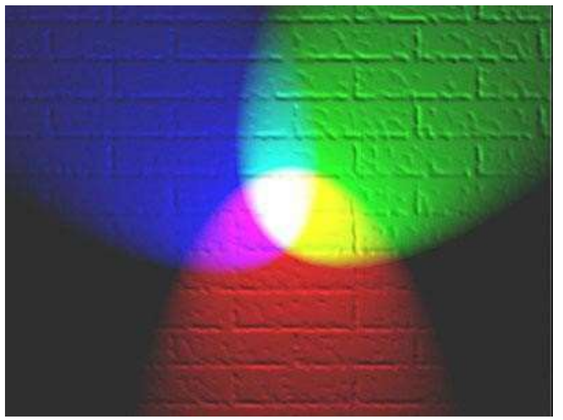
Additive Color

Primary colors used **(RGB)**

- **Red**
- **Green**
- **Blue**.

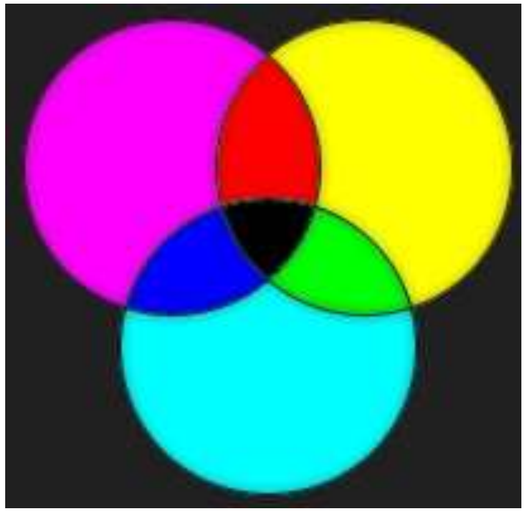


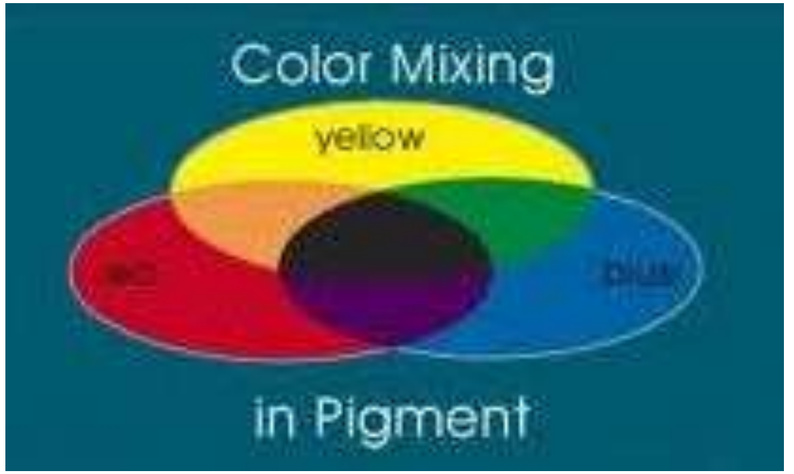
- CRT is an example of **additive color** where primary colors are added together to give the perceived color.



Subtractive Color

- It suits for processes such as **commercial printing and painting** as white surface is used as background.
- The primaries are usually the complementary colors (**CMY**)
  - **Cyan**
  - **Magenta**
  - **Yellow**





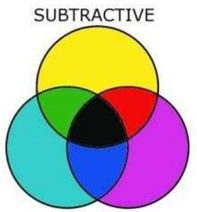
RV College of Engineering

Differences between Additive color and Subtractive color

Go, change the world

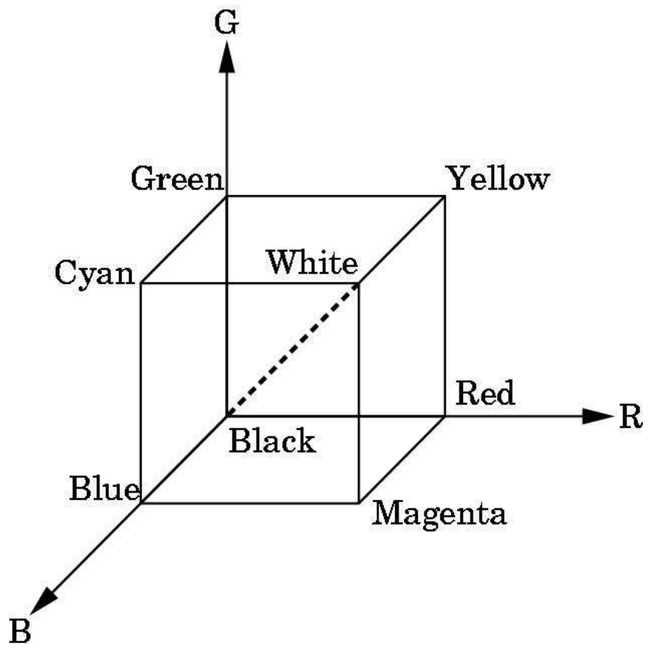
Additive Color	Subtractive Color
Used in CRTs.	Used for commercial printing and painting.
Primary colors are Red, Green, Blue (RGB).	Primary colors are Cyan, Magenta, Yellow (CMY).
Primaries add light to an initially **black** display to give desired color.	Primaries subtract color from an initially **white** display to give desired color.



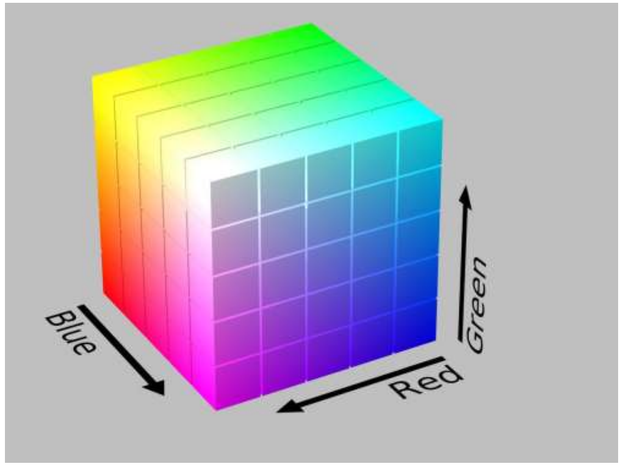


We can view color as a point in a color solid

The vertices of a cube correspond to:



- **black**(no primaries on)
  - **red**, **green**, **blue**(one primary fully on)
    cyan(green and blue fully on)
    magenta(red and blue fully on)
  - **yellow**(red and green fully on)
    white(all primaries fully on)



- **All colors along principal diagonal line have equal tristimulus values and appear as shades of gray.**

Color Models

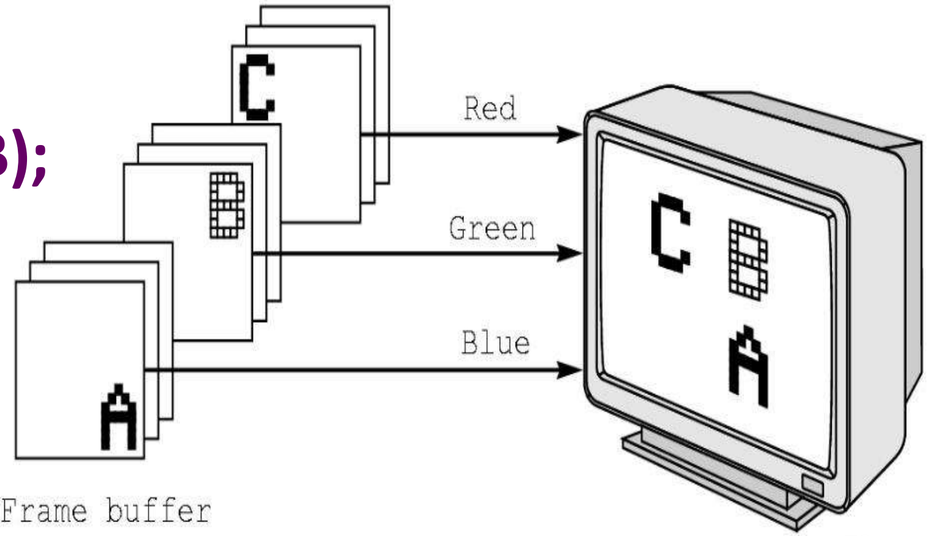
(Dec 2011/ 12 marks, June 2012/6 marks)

There are 2 color models

1. **RGB color model**
2. **Index color model**

- In **additive color** RGB system, there are **separate buffers** for Red, Green and Blue images.
- Each pixel has separate R, G, and B components that corresponds to location in memory.
- **Ex : 24 bits for each pixel will be divided into**
  - 8 bits + 8 bits + 8 bits = 24 bits

    R G B
- In OpenGL, color wrt RGB is represented as
  - **void glColor3f(GLfloat R, GLfloat G, Glfloat B);**



RV College of Engineering

RGB Color

Go, change the world

- In OpenGL, the color attribute can be specified as a number between 0.0 and 1.0
- Ex : to draw in red, the following function call is used:

  glColor3f(1.0, 0.0, 0.0); Default value of each

  one is 0.
- The execution of this function will set the present drawing color to red. Because the color is part of the state, draw in **red is continued until the color is changed.**
- The "**3f**" is used in a manner similar to the **glVertex** function:
- It conveys that, a three-color (RGB) model is used, and that the values of the components are given as floats in C.

RGBA Color

- Four color model namely RGBA system.
- A(alpha) – used for creating ***fog effects***(combining the images).
  - – transparent (passes all kinds of light)
  - – opaque(does not pass any light)

- **glClearColor(1.0,1.0,1.0,1.0)**
- would set the background color of the window to white and opaque.

1. RGBA = Red + Green + Blue + Alpha

It's 4 values that describe a color and how transparent it is.

R	G	B	A
Red	Green	Blue	Alpha
(0.0 – 1.0)	(0.0 – 1.0)	(0.0 – 1.0)	(0.0 – 1.0)

2. What is Alpha?

Alpha controls transparency.

A = 0.0	A = 0.5	A = 1.0
		
Fully transparent (see through)	Half transparent (slightly see through)	Fully opaque (cannot see through)

3. The Command

```
glClearColor( 1.0 , 1.0 , 1.0 , 1.0 );
```

1.0	1.0	1.0	1.0
R	G	B	A
(Red)	(Green)	(Blue)	(Alpha)

4. What does it mean?

- R = 1.0 → maximum red
- G = 1.0 → maximum green
- B = 1.0 → maximum blue
- A = 1.0 → fully opaque

So the window background color is **WHITE** and it is **OPAQUE.**



Result: White background (not transparent)

Indexed Color

- Commonly used color model.
- It is used when **frame buffer has limited depth** *(No. of bits per pixel).*
- ***Example :***
  - If the frame buffer has 1280 x 1024 array of pixels but with each pixel consisting of only 8 bits, then color index model is used.

Input	Red	Green	Blue
0	0	0	0
1	$2^m - 1$	0	0
.	0	$2^m - 1$	0
.	.	.	.
.	.	.	.
$2^k - 1$	.	.	.
	m bits	m bits	m bits

Why Indexed Color?

Useful when the frame buffer has limited bits per pixel.

Frame buffer

Memory that stores information for screen pixels.

More pixels + more bits = more memory.

Limited depth

Depth here means the number of bits available for each pixel.

Example: 8 bits per pixel.

Indexed solution

Store a small number (index) for each pixel and keep the RGB colors in a table.

8 bits per pixel → 2^8 = 256 possible index values



So an 8-bit indexed system can select from up to 256 table entries.

RV College of Engineering

Indexed Color

In the model, suppose there are ‘k’ bits per pixel, then each pixel value is an integer between 0 to $2^k - 1$.

Input	Red	Green	Blue
0	0	0	0
1	$2^m - 1$	0	0
.	0	$2^m - 1$	0
.	.	.	.
.	.	.	.
$2^k - 1$	.	.	.
	m bits	m bits	m bits

- We can display colors with a precision of m bits.
- We can choose from 2^m reds, 2^m greens and 2^m blues.

- `glIndexi(element);`
- `glutSetColor(int color, Glfloat red, Glfloat reen, Glfloat blue);`

What does the RGB table mean?

The index selects an RGB combination.

INDEX → R + G + B → FINAL COLOR

0	0	0	0	**BLACK**
1	MAX	0	0	**RED**
2	0	MAX	0	**GREEN**
3	0	0	MAX	**BLUE**

Key point

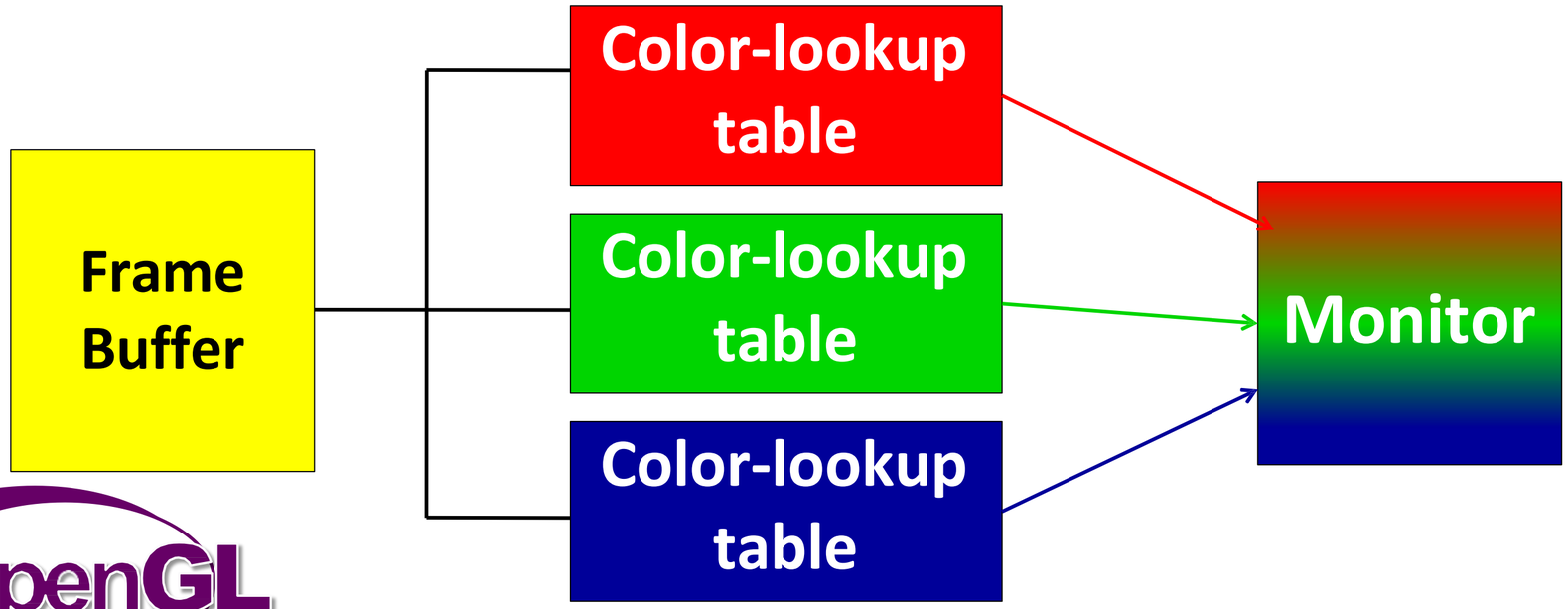
The table is simply a lookup: the index tells the computer which RGB color to display.

RV College of Engineering

Go, change the world

Indexed Color

Input	Red	Green	Blue
0	0	0	0
1	$2^m - 1$	0	0
.	0	$2^m - 1$	0
.	.	.	.
.	.	.	.
$2^k - 1$	.	.	.
	m bits	m bits	m bits



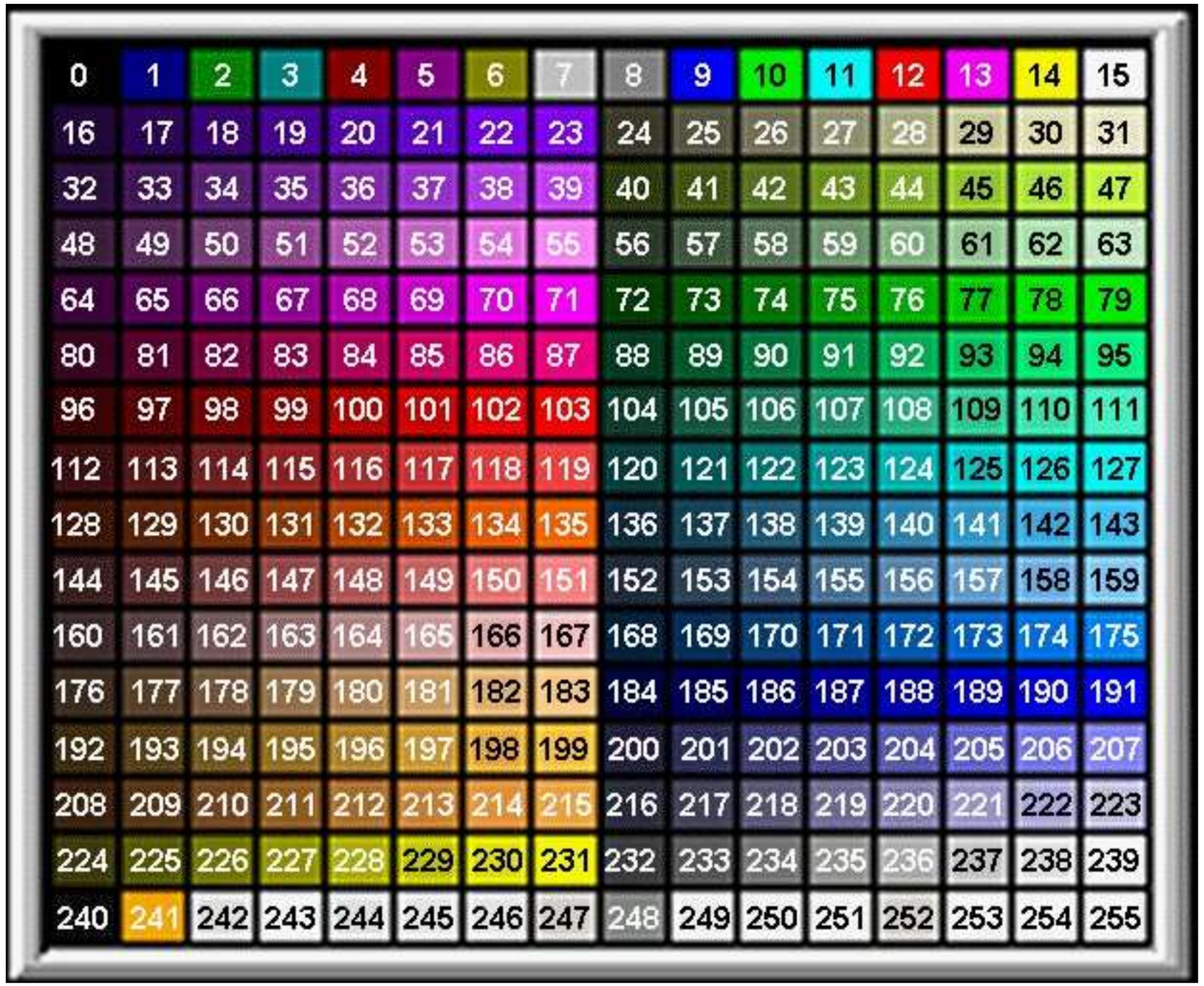
RV College of Engineering

Go, change the world

Indexed Color palette

1280 × 1024 = 1,310,720 pixels

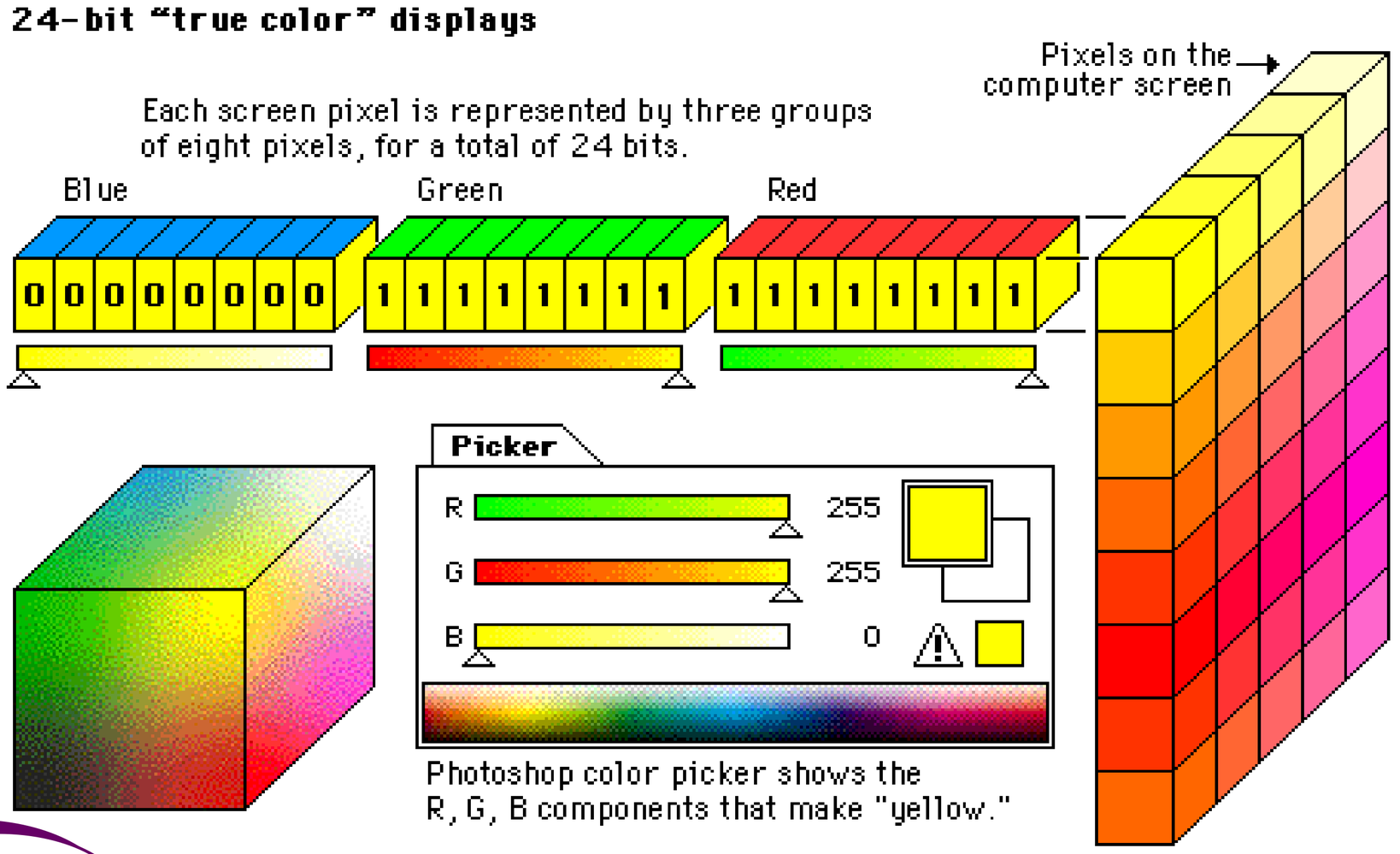
If k=8 bits per pixel,
we get $2^8 - 1 = 256$ colors palette.



RV College of Engineering

Go, change the world

Indexed Color palette



One-minute recap

The complete concept in five points.

1	**Normal RGB**	Pixel directly stores R + G + B.
2	**Indexed Color**	Pixel stores only an index number.
3	**Color Table**	Index points to the corresponding RGB values.
4	**8 bits**	$2^8 = 256$ possible indexes.
5	**Why use it?**	Useful when memory / bits per pixel are limited.

Remember this flow
PIXEL → INDEX → COLOR TABLE → RGB → SCREEN

Indexed Color

Advantages

- We can operate with such systems where the **number of bits per pixel is less.**
- The programmer can change the lookup table according to his requirements and hence has the flexibility to enjoy the usage of all possible colors.

Disadvantages

- All possible colors would not be available for usage simultaneously.

CONTROL FUNCTIONS

RV College of Engineering

Go, change the world

Control Functions

Control functions enable the programmer to communicate with the window system, initialize the program etc.

1. glutInit(int *argcp, char **argv)
2. glutCreateWindow(char *title)
3. glutInitDisplayMode()
4. glutInitWindowSize()
5. GlutInitWindowposition()
6. glutDisplayFunc()
7. glutMainLoop()

RV College of Engineering

Control Functions

Go, change the world

1. **glutInit(int *argcp, char **argv)**
   - It initiates an interaction between the windowing system and OpenGL. It is used before opening a window in the program.
   - The two arguments allow the user to pass command-line arguments, as in the standard C main function, and are usually the same as in main.

2. **glutCreateWindow(char *title)**
   - It opens an OpenGL window.
   - The title string provides title at the top to the window displayed.

Control Functions

3. **glutInitDisplayMode()**
   Used to initialize the display window created, by specifying color model, buffering used etc.

4. **glutInitWindowSize()**
   Used to specify the size of the window to be created.

5. **glutInitWindowPosition()**
   Used to specify the position of the created window on the monitor with respect to the top left corner.

Control Functions

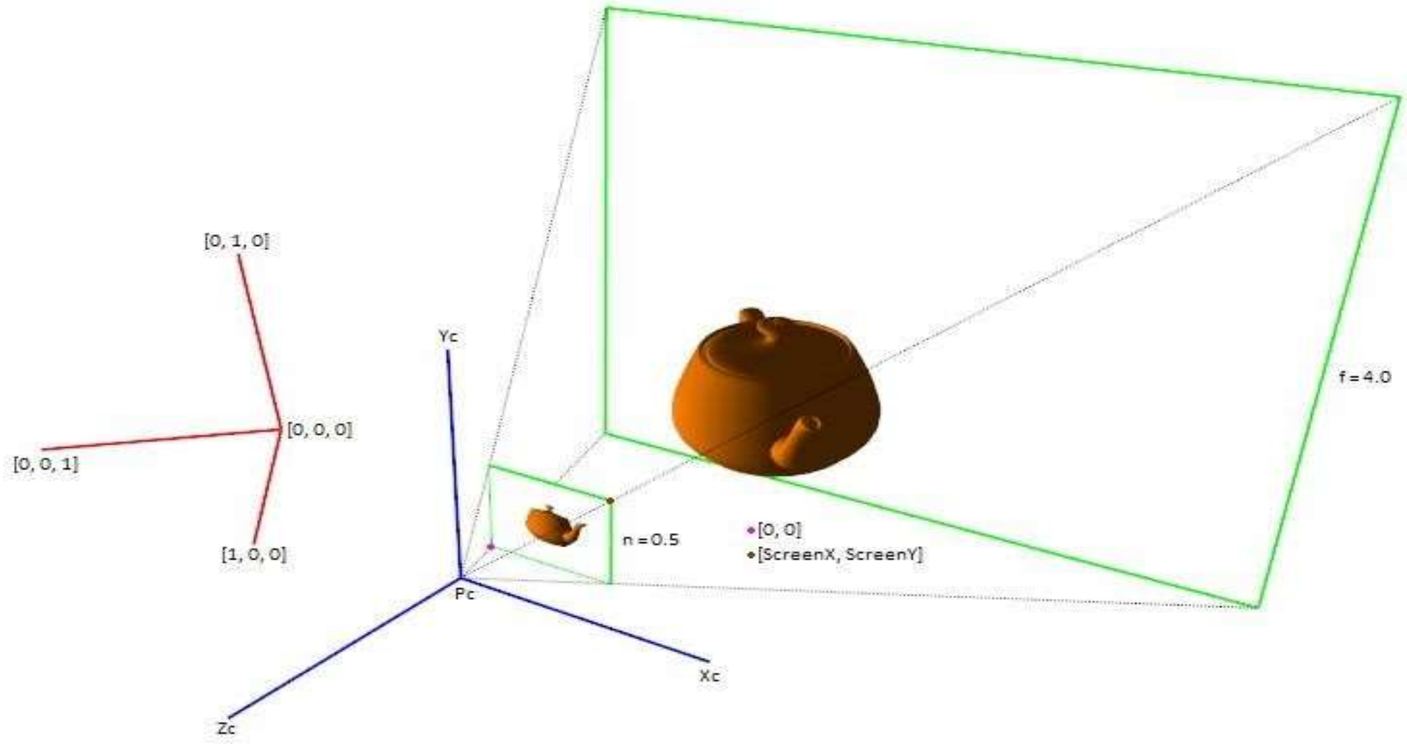
6. glutDisplayFunc()

- Used to specify the display call back.
- This function executes when the window is created for the first time.
- It is also called when the window is moved from one location on the screen to another.

7. glutMainLoop()

- Causes the program to begin an event processing loop.
- If there are no events to process, then the program would enter the wait state with the output on the screen.
- It is similar to getch() in C program.

Viewing



Viewing

- Viewing is a type of graphics functions which enables the programmer to specify the view facilities such as top view, front view, left side view, right side view, zoom in and zoom out.
- Determines the appearance of the object on the output device.
- If the programmer does not specify a view explicitly, **default one(orthogonal view) is considered.**

Viewing can be

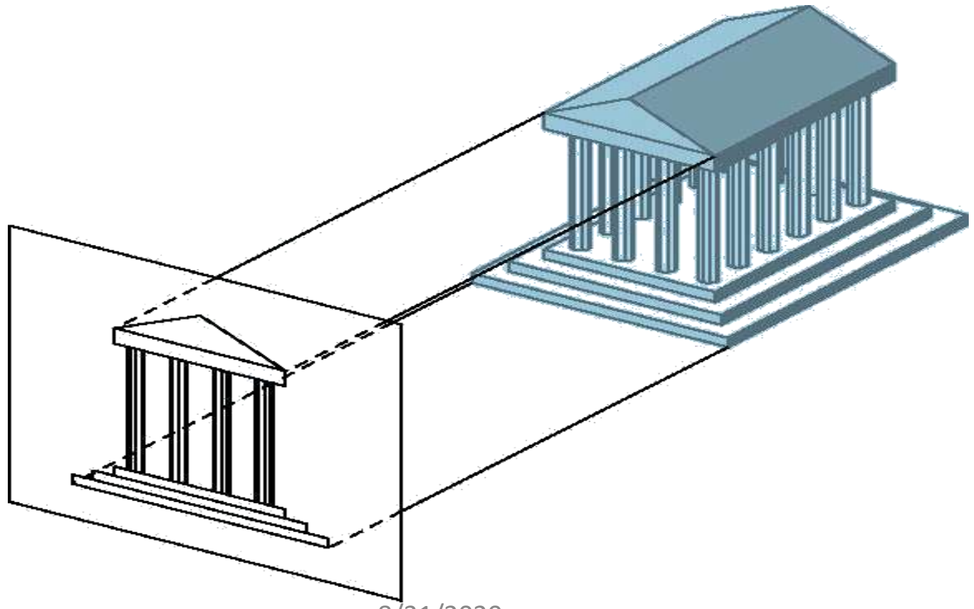
- Orthographic view
- 2D view
- Matrix mode

RV College of Engineering

Orthographic View (July 2009)

Go, change the world

- Simplest and OpenGL's **default view** is orthographic projection
- Can be obtained in the synthetic camera model by placing the camera at an **infinitely far distance** from the object.
- The projections(lines) are perpendicular(orthogonal) to the projection plane and hence the projection is called as orthogonal projection/viewing.



OpenGL

RV College of Engineering

Orthographic View

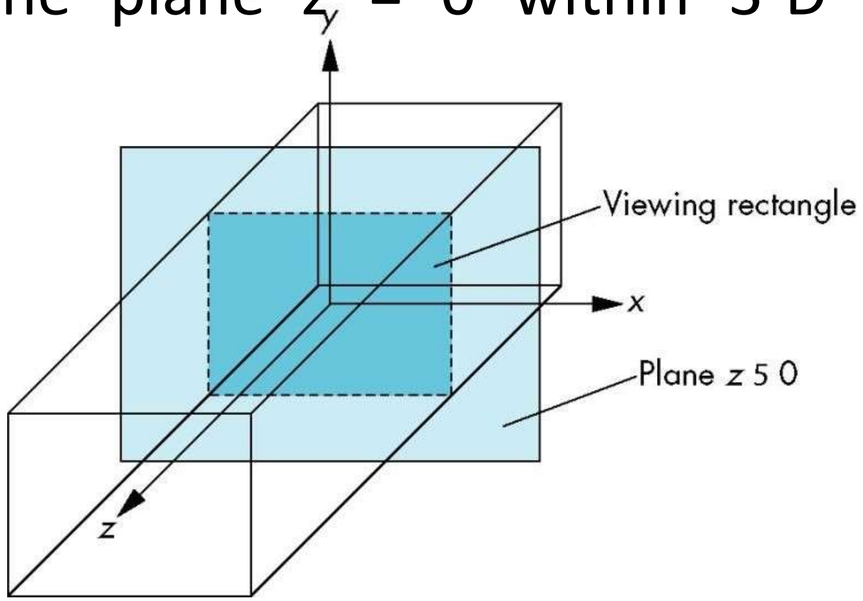
Go, change the world

- In OpenGL, an orthographic projection with a right parallel piped viewing volume is specified via the following
- **void glOrtho(Gldouble left, Gldouble right, Gldouble bottom, Gldouble top, Gldouble near, GLdouble far)**
- Example: glOrtho(0, 500, 0, 500, -1, 1);
- Show the coordinate area from **0 to 500 in X** and **0 to 500 in Y**, without perspective.
- Sees only those objects in the volume specified by the viewing volume. It can also include objects that are behind the camera!

RV College of Engineering

Orthographic View

- If viewing volume is not specified, OpenGL uses its default, a 2 x 2 x 2 cube, with the origin in the center. In terms of 2-D plane, the bottom-left corner is at(-1.0, -1.0), and the upper-right corner is at (1.0, 1.0).
- 2-D graphics view is a special case of 3-D graphics. Hence the viewing rectangle is in the plane $z = 0$ within 3-D viewing volume:

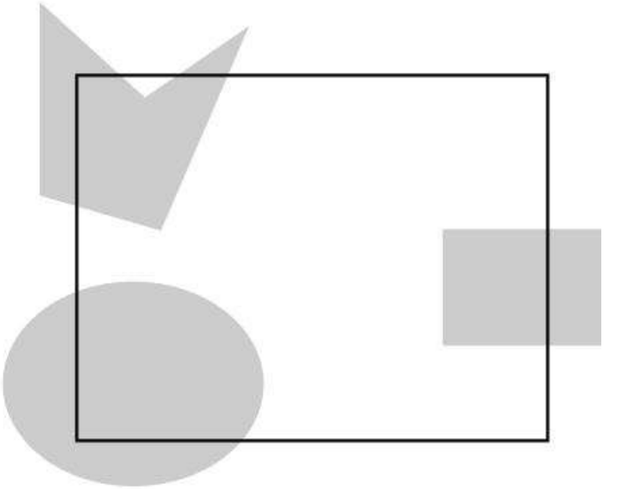


RV College of Engineering

Go, change the world

2-D viewing

- ***2-D viewing:*** is based on taking a rectangular area of 2-D world and transferring its contents to the display.
- The area of the world of which the image is to be taken is known as the *viewing rectangle* or *clipping rectangle*.
- Objects inside the rectangle are in the image; objects outside are clipped out and are not displayed.





2-D viewing

- **void gluOrtho2D(GLdoub1e left, GLdouble right, GLdouble bottom, GLdouble top)**
- in the utility library may make the program more consistent with its 2-D structure. This function is equivalent to **glOrtho** with near and far set to -1.0 and 1.0, respectively.

Matrix Modes in OpenGL

- Matrix mode tells OpenGL which matrix we are currently working with.
- Think of it as choosing the correct notebook before writing.
- Main modes used in basic OpenGL: GL_PROJECTION and GL_MODELVIEW.

1. GL_PROJECTION

- Code: glMatrixMode(GL_PROJECTION);
- Meaning: “I am setting how the scene will be viewed.”
- Used for projection settings such as Orthographic and Perspective views.
- Example: glOrtho(0, 500, 0, 500, -1, 1);

2. GL_MODELVIEW

- Code: glMatrixMode(GL_MODELVIEW);
- Meaning: “I am working with the objects and their position.”
- Used for moving, rotating, and scaling objects.
- It can also be used for setting the camera/view transformation.

Why do we need two modes?

PROJECTION → How do we see the scene?

MODELVIEW → Where is the object and how is it transformed?

Simple memory trick: Projection = VIEW; ModelView = OBJECT.

Typical OpenGL Code

```
glMatrixMode(GL_PROJECTION);
glLoadIdentity();
glOrtho(0, 500, 0, 500, -1, 1);

glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
```

- glMatrixMode() does NOT itself move or rotate an object.
- It only tells OpenGL which matrix subsequent matrix operations should affect.
- GL_PROJECTION → viewing/projection setup
- GL_MODELVIEW → object/camera transformations

RV College of Engineering

Go, change the world

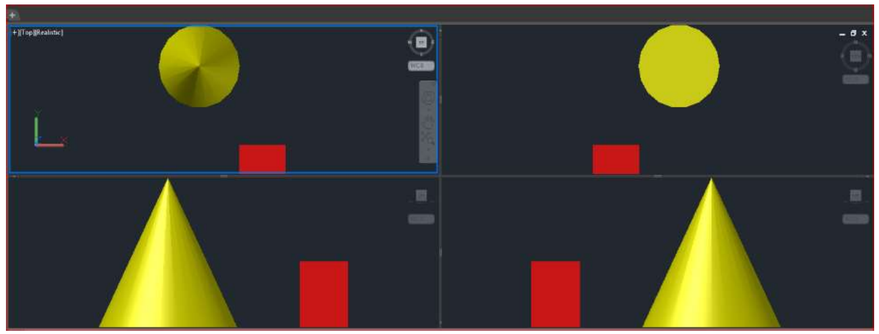
Aspect Ratio and Viewports

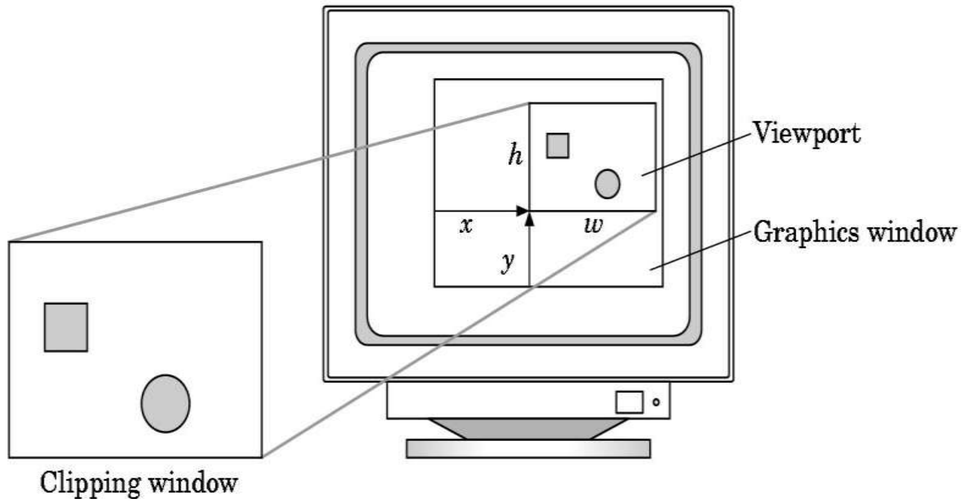
(July 2013/4 marks)

Aspect ratio : ratio of the rectangle's width to its height.

Viewport : Rectangular area of the display window.

By default, it is the entire window, but can be set to any smaller size in pixels through function.





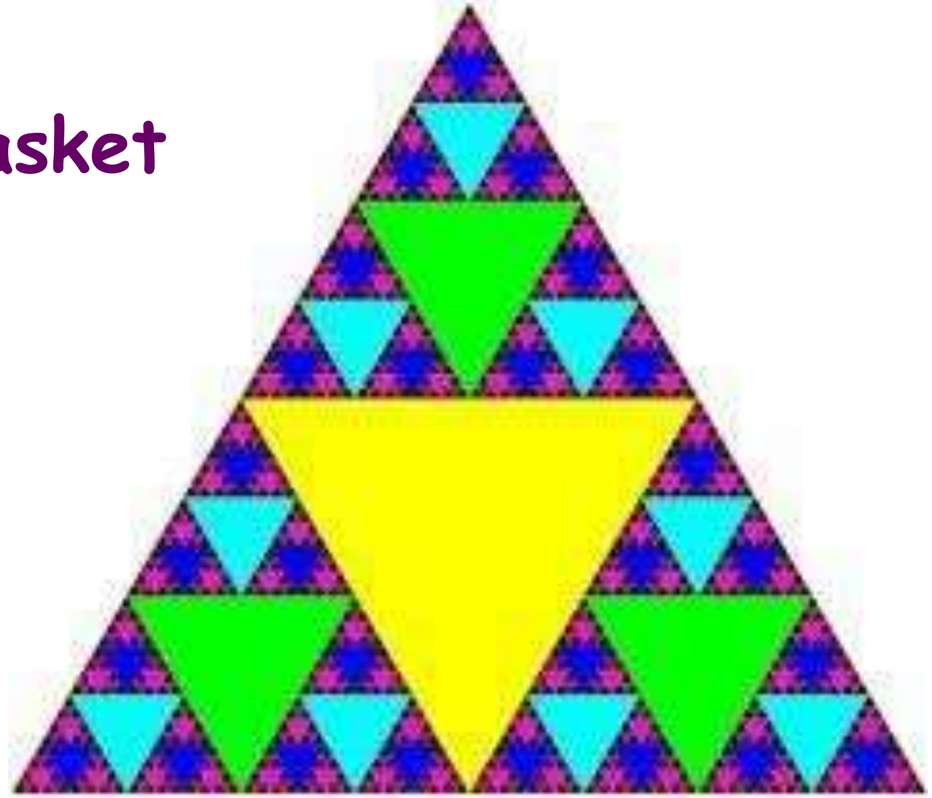
Aspect Ratio and Viewports

void glViewport(GLint x, GLint y, GLsizei w, GLsizei h)

- **(x,y)**
  Specify the lower left corner of the viewport rectangle, in pixels.
  The initial value is (0,0).

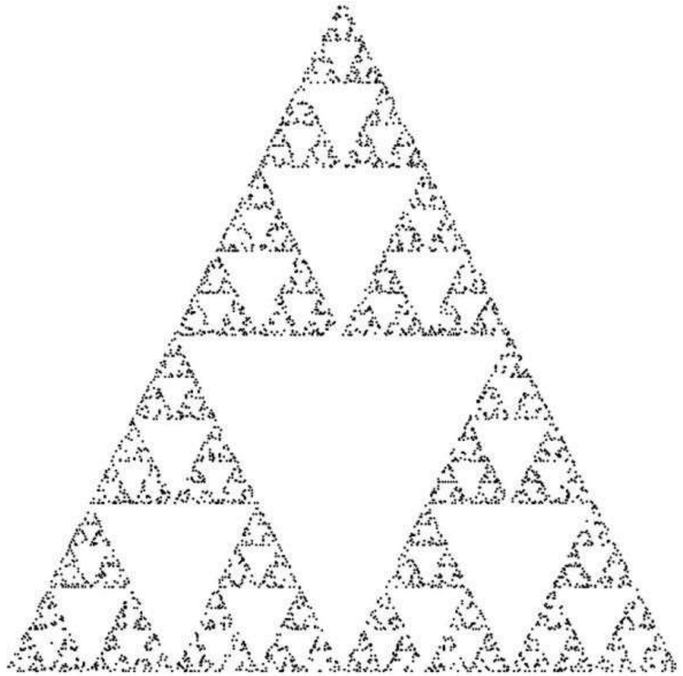
- **w and h (width and height)**
  Specify the width and height of the viewport.
  When a GL context is first attached to a window, width and height are set to the dimensions of that window.

Sierpinski Gasket Program

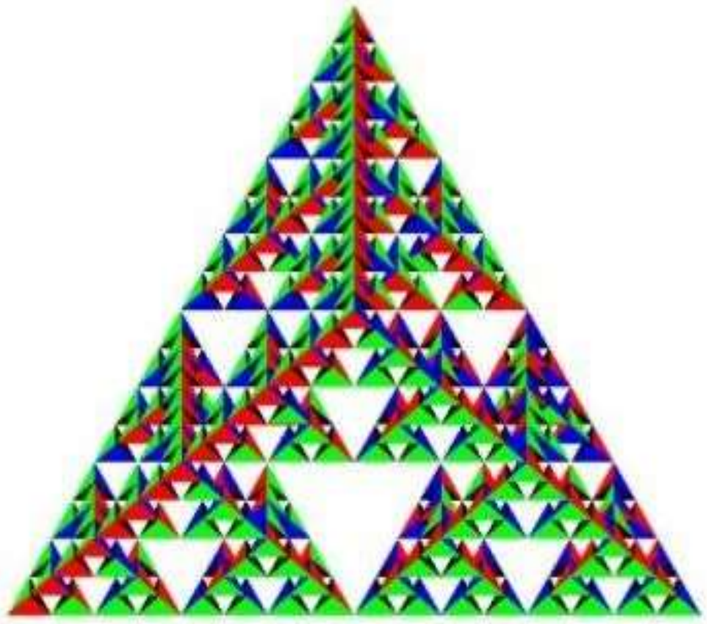


Sierpinski Gasket Construction

random number generation



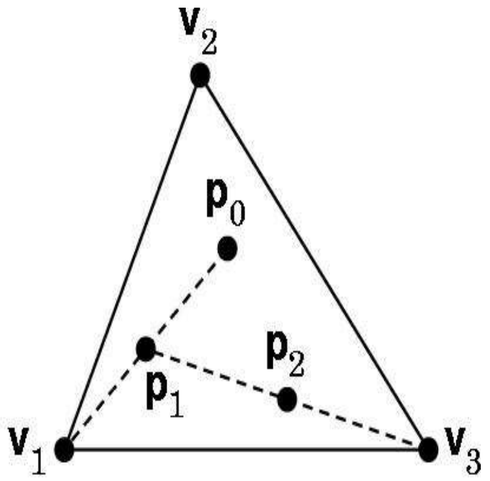
recursion



Construction of Sierpinski Gasket - using random numbers

The construction of sierpinski gasket as follows:

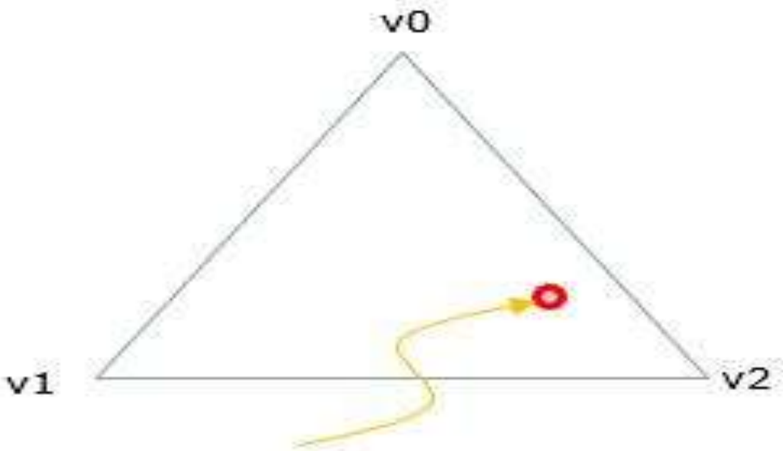
1. Pick up a random point inside the triangle.
2. Select one of the three vertices at random.
3. Find the point halfway between initial point and the randomly selected vertex.
4. **Display this new point** by putting some sort of marker, such as a small circle, at its location.
5. Replace the initial point with this new point.
6. Return to step 2.



Thus, each time a point that is generated, it is displayed on the output device.

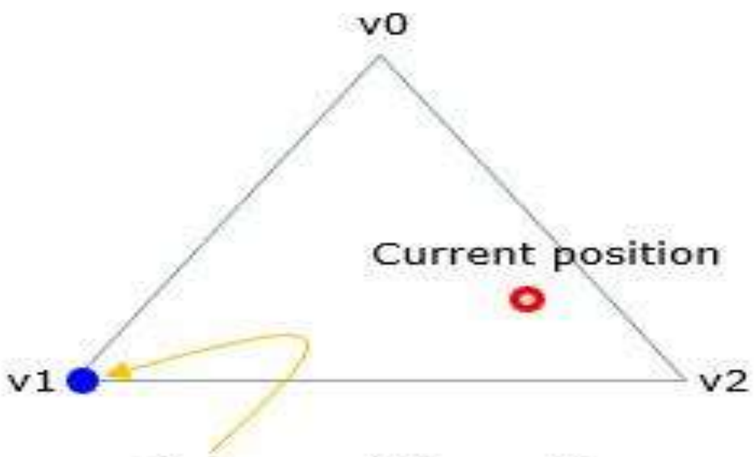
In the figure p_0 is the initial point, and p_1 and p_2 are the first two points generated by the algorithm.

Step 1



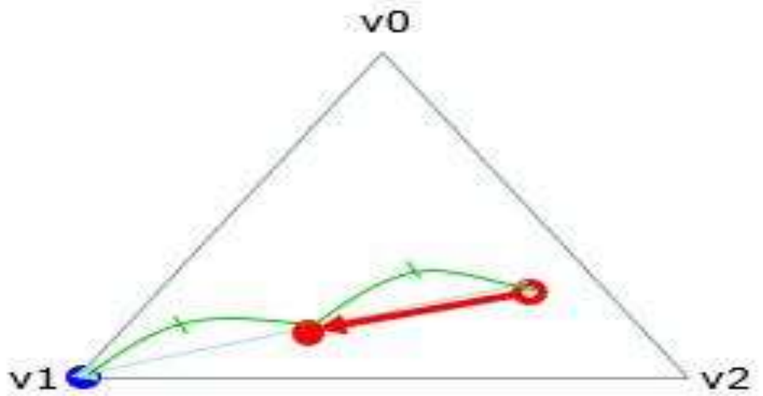
Initial position
(you can pick any point within the triangle)

Step 2



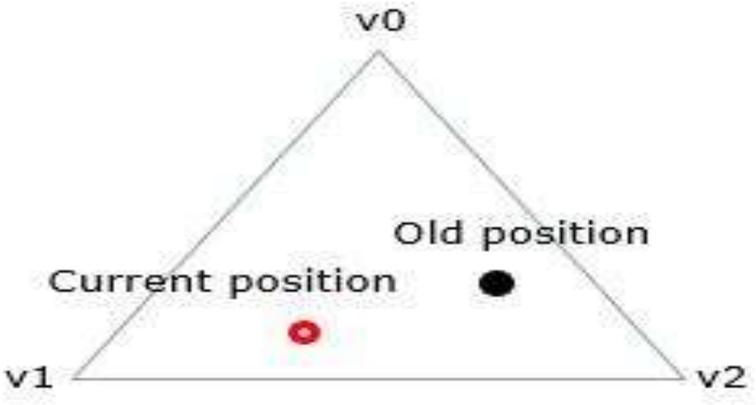
Pick one of the vertices randomly
(Let's assume that v1 is picked/selected)

Step 3



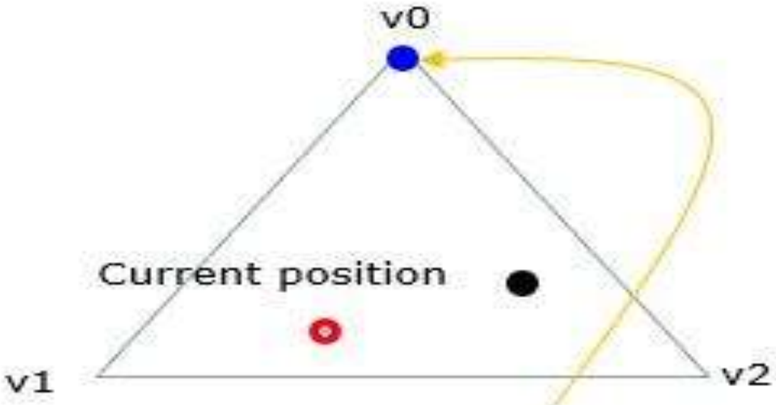
Move to the half way towards the selected vertex (v1 in this example)

Step 4



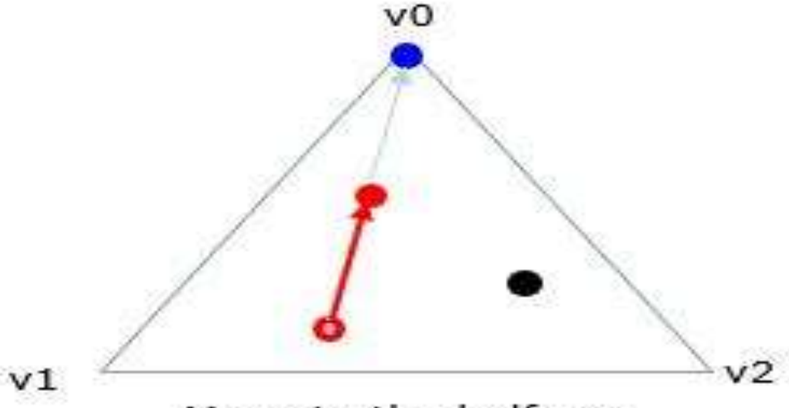
Now take this point (moved point) as a current position

Step 5

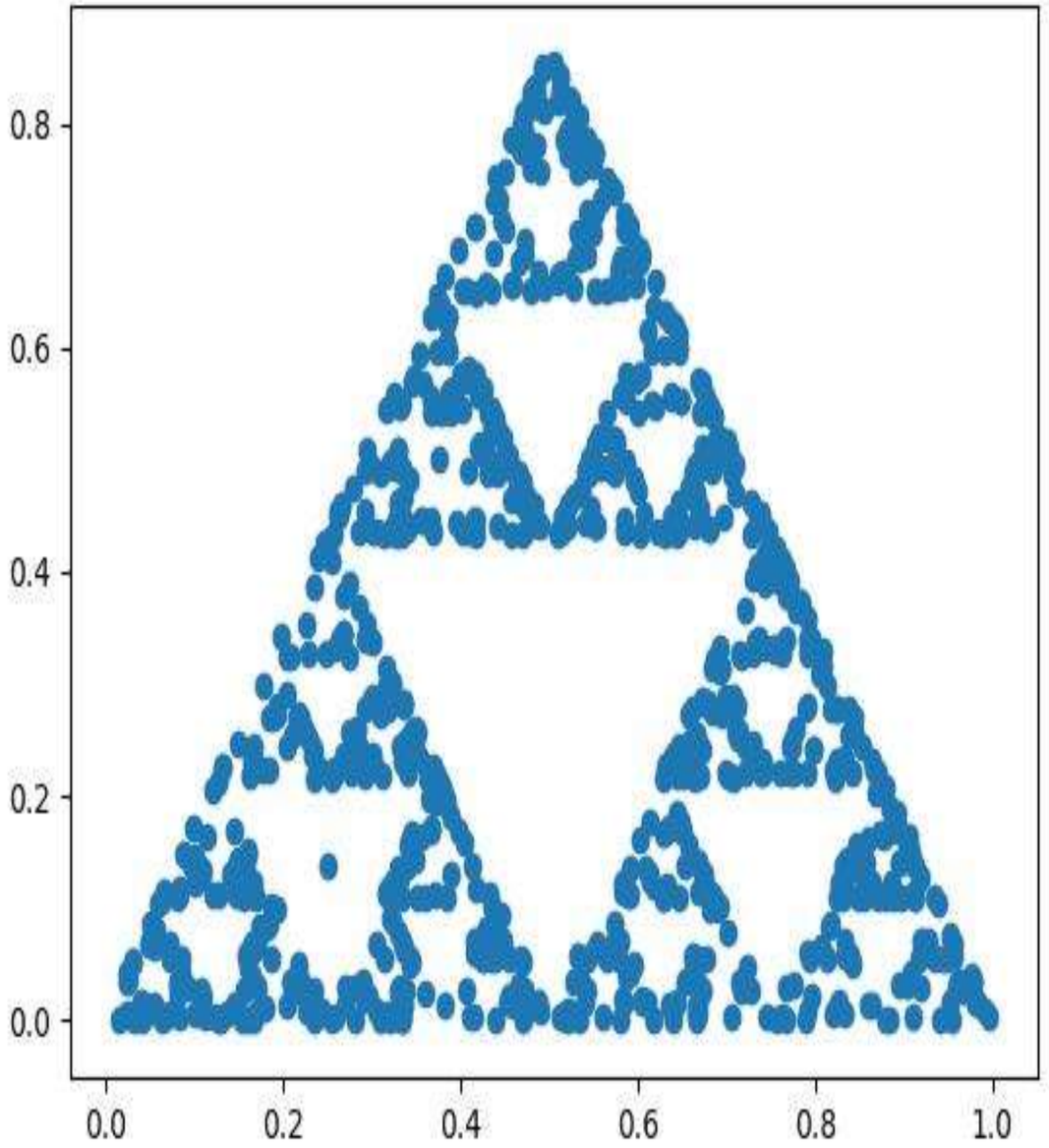


Pick one of the vertices randomly
(Let's assume that v0 is picked/selected)

Step 6

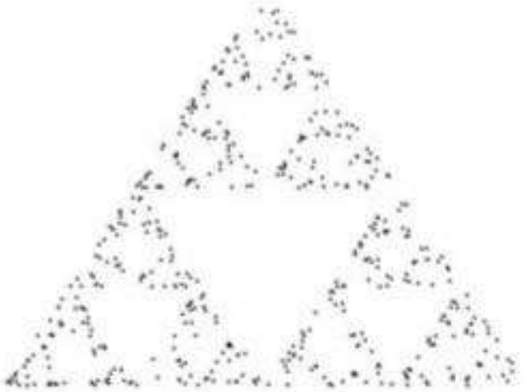


Move to the half way towards the selected vertex (v0 in this example)

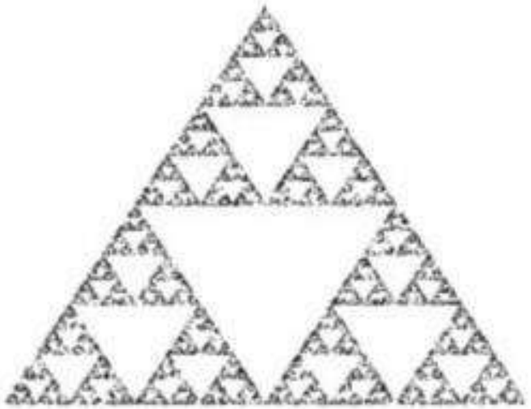




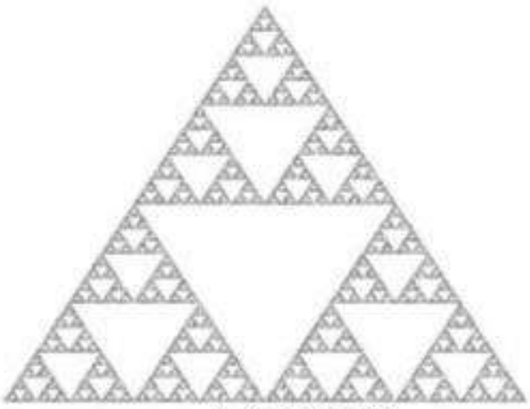
$n = 50$



$n =$



$n = 5000$



$n = 50000$

Construction of Sierpinski Gasket - using random numbers

Possible form of our program might be:

```
main ( )
{
        initialize_the_system ();
        for (some_number_of_points )
        {
              pt = generate_a_point();
              display_the_point(pt);
        }
        cleanup()
}
```

Construction of Sierpinski Gasket using OpenGL

```
#include <GL/glut.h>
void main (int argc, char **argv) /* the main function */
{
        glutInit(&argc,argv);
        glutDisplayMode(GLUT_SINGLE | GLUT_RGB);
        glutInitWindowSize (500,500);
        glutInitWindowPosition(0,0);
        glutCreateWindow("simple OpenGL example");
        glutDisplayFunc(display);
        myinit( );
        glutMainLoop( ); // Event Processing
}
```

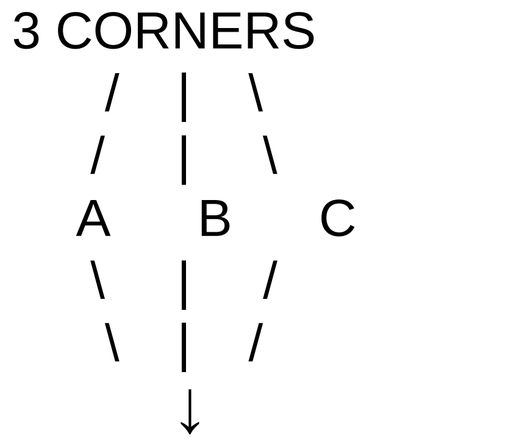
Construction of Sierpinski Gasket using OpenGL

```
typedef GLfloat point2 [2];
/* defines a point as two dimensional array */

void display(void)
{       /* an arbitrary triangle */
        point2 vertices[3]= { {0.0,0.0}, {250.500}, {500.0,0.0}};
        static point2 p = {75.0, 50.0}; /* set to any initial point */
        int j,k;
        int rand( ); /* standard random number generator */
```

Construction of Sierpinski Gasket using OpenGL

```
for (k=0;k<5000;k++)
{       j = rand ( )%3; /* pick a random vertex from 0, 1, 2 */
        p[0] = (p[0]+ vertices[ j ][0])/2;
        p[1] = (p[1]+ vertices[ j ][1])/2;
        /* display the new point */
        glBegin(GL_POINTS);
        glVertex2fv(p);
        glEnd();
}
glFlush(); /* points are to be displayed as soon as possible glFlush ensures
that points are rendered to    the screen as soon as possible */
}
```



↓

Choose randomly

↓

One corner

↓

Move HALF way

↓

New point

↓

Draw point

↓

Repeat

↓

5000 times

↓

SIERPINSKI GASKET

Sierpinski Gasket - output

3 CORNERS

A B C

↓

Choose randomly

↓

One corner

↓

Move HALF way

↓

New point

↓

Draw point

↓

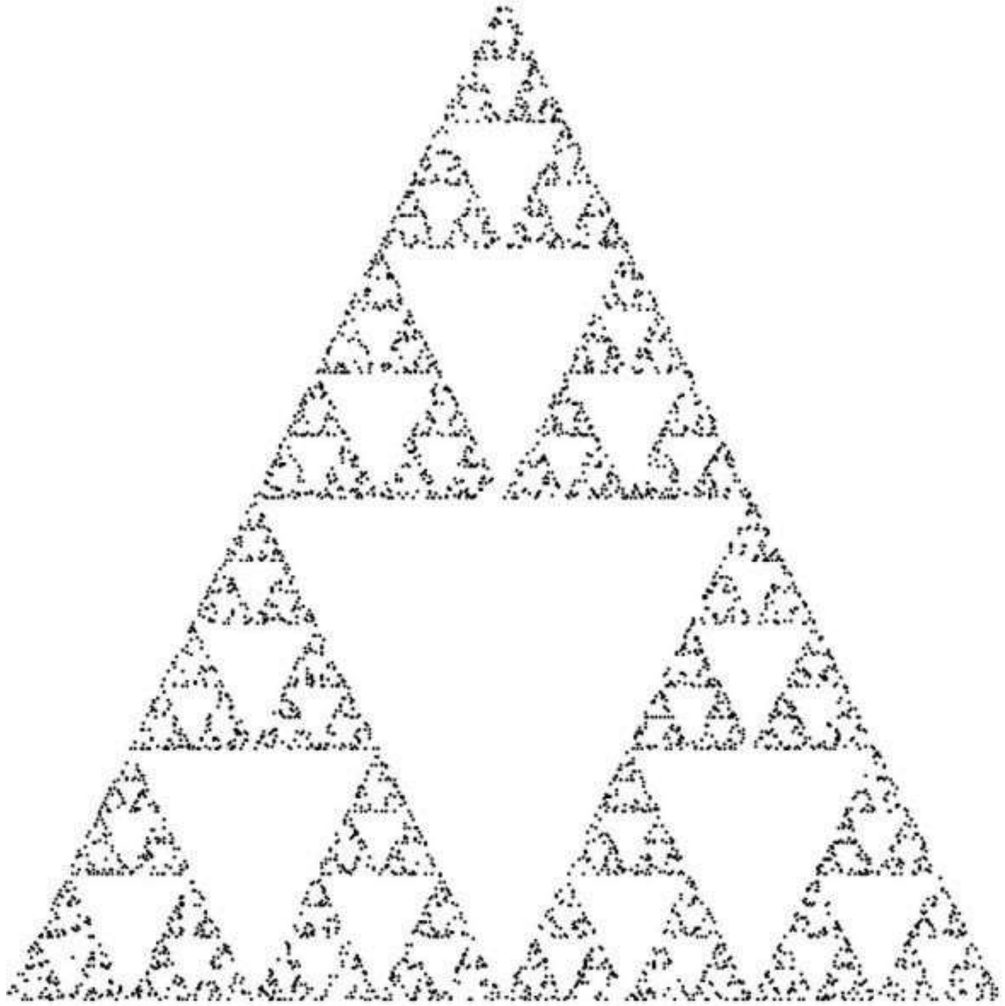
Repeat

↓

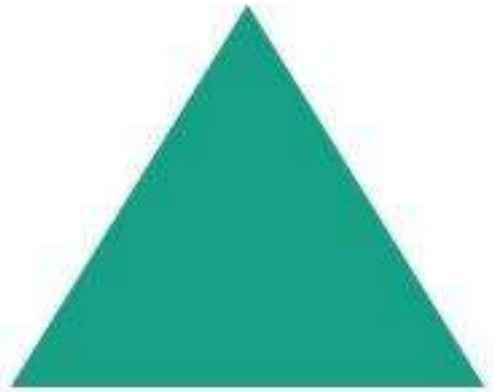
5000 times

↓

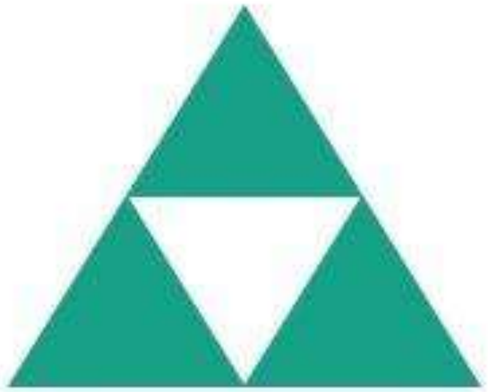
SIERPINSKI GASKET



Construction of Sierpinski Gasket using Recursion



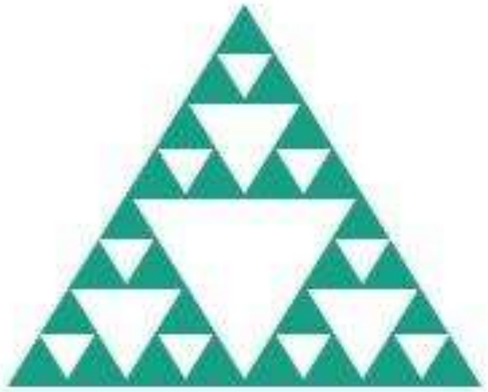
Step 1



Step 2



Step 3



Step 4

Step 1 → 1 triangle

Step 2 → 3 triangles

Step 3 → 9 triangles

Step 4 → 27 triangles

Step 5 → 81 triangles

PROGRAM 3

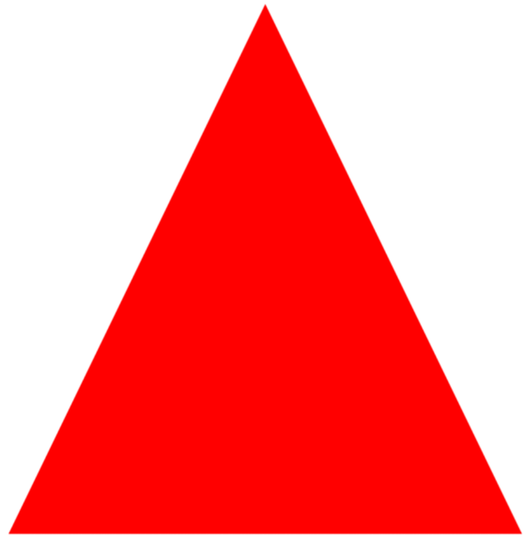
Write a program to recursively subdivides a tetrahedron to form 3D Sierpinski gasket. The number of recursive steps is to be specified at execution time.

3D Sierpinski gasket

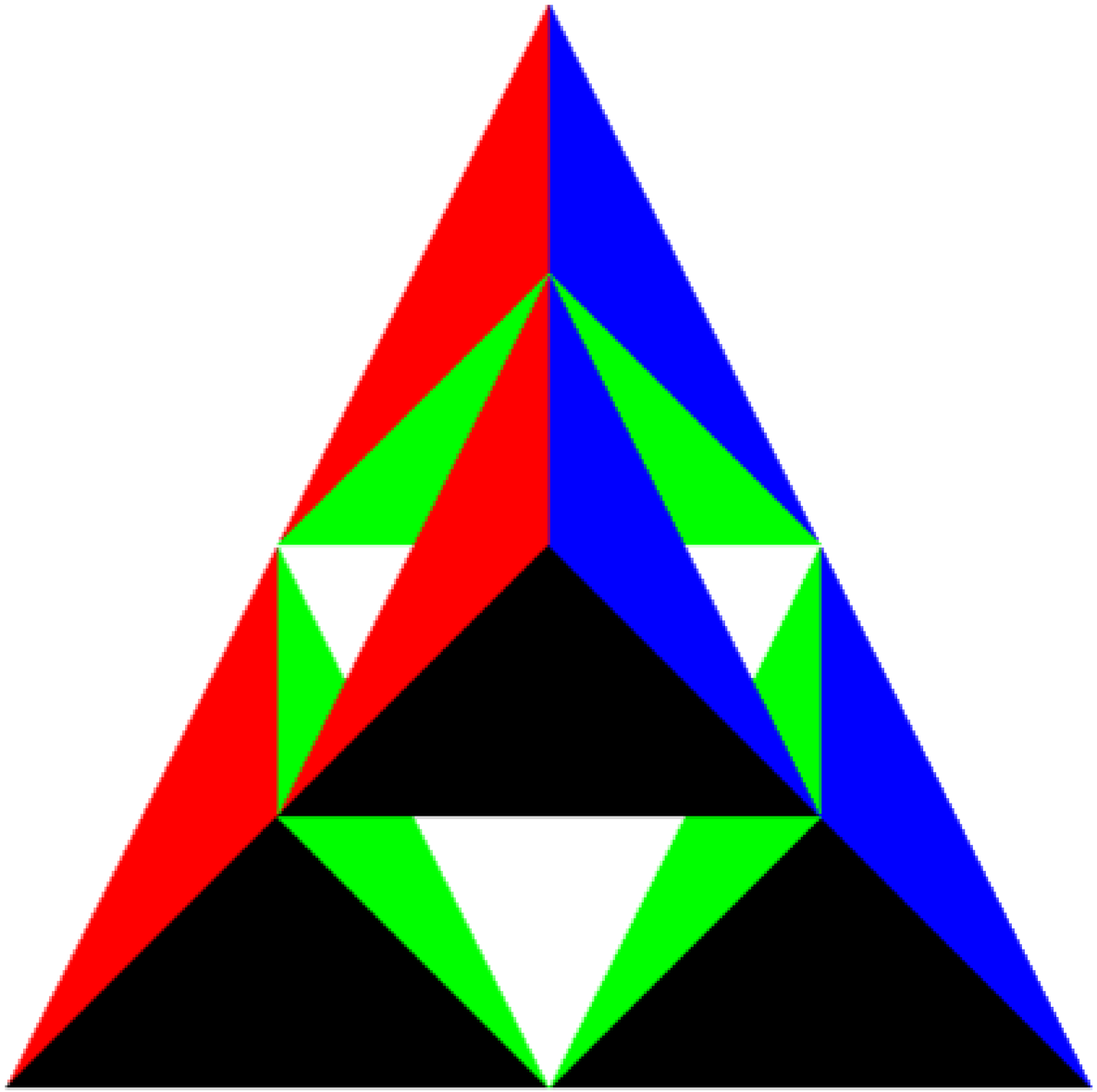
- A fractal which can be constructed by a recursive procedure; at each step a triangle is divided into four new triangles.
- The central triangle is removed and each of the other three treated as the original was, and so on, creating an infinite regression in a finite space.
- **Origin:**

  1970s: named after Wacław Sierpiński (1882–1969), Polish mathematician
- **Application :**

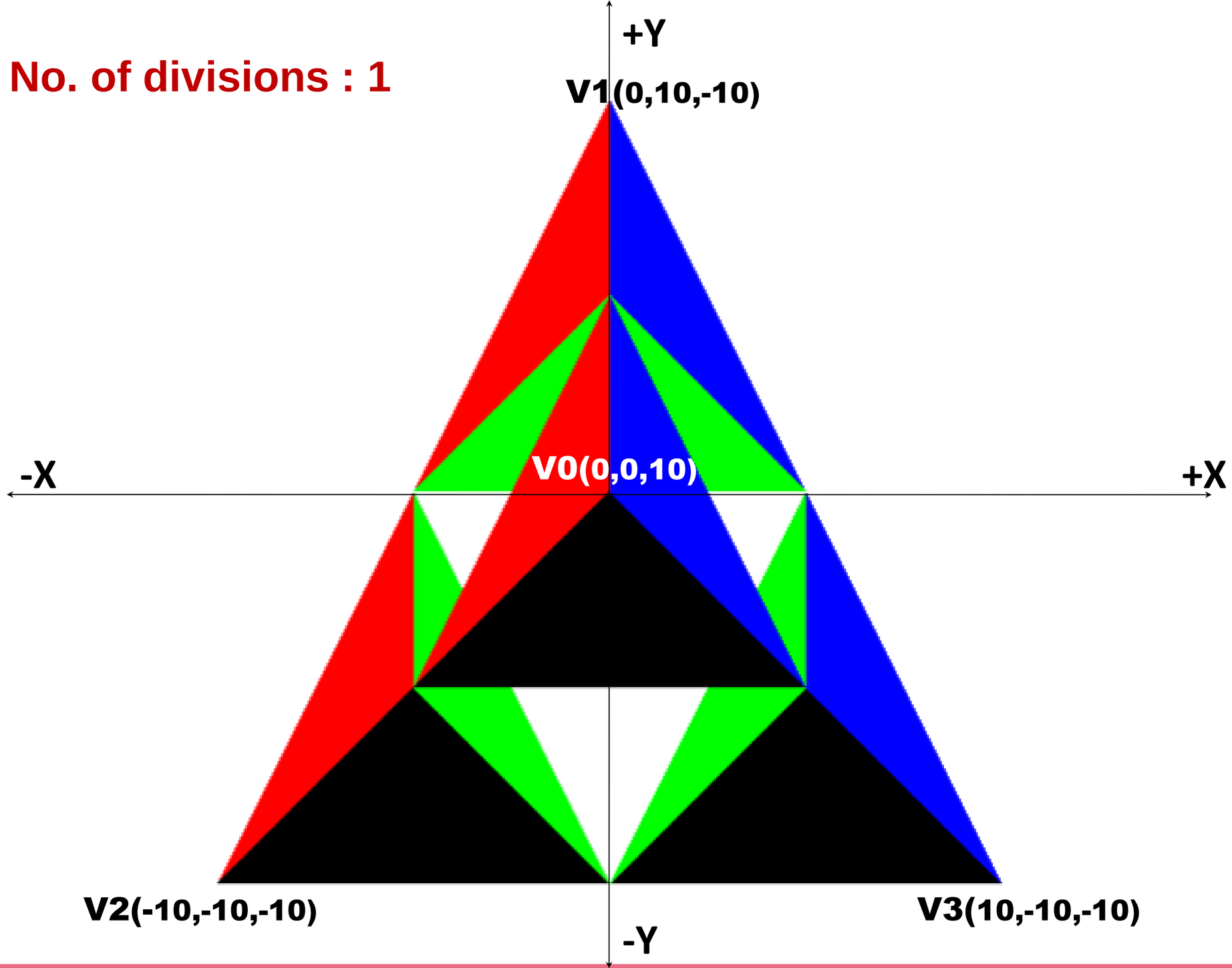
  Fractal geometry



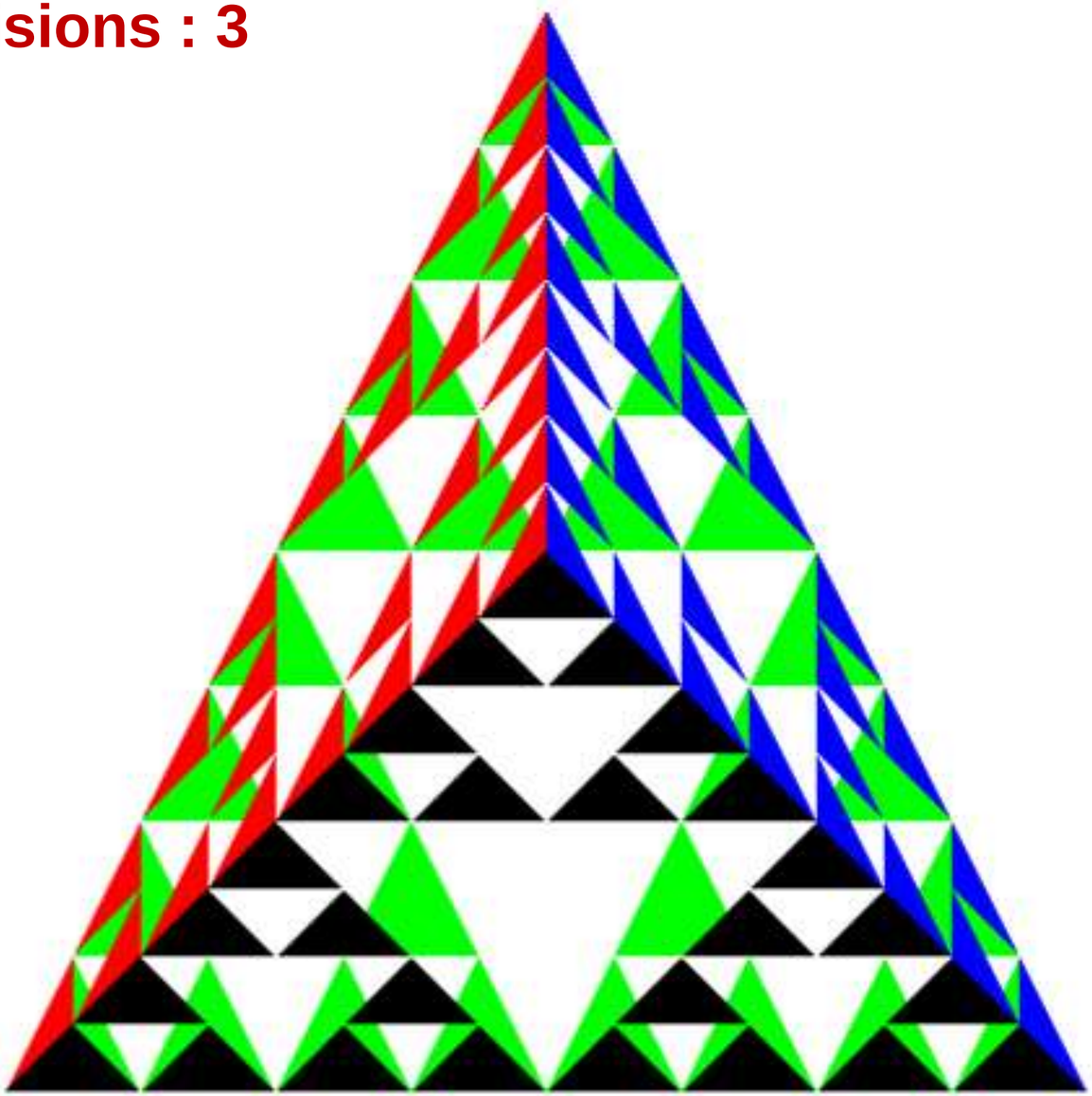
No. of divisions : 1



No. of divisions : 1



No. of divisions : 3



```
#include <stdio.h>
#include <GL/glut.h>

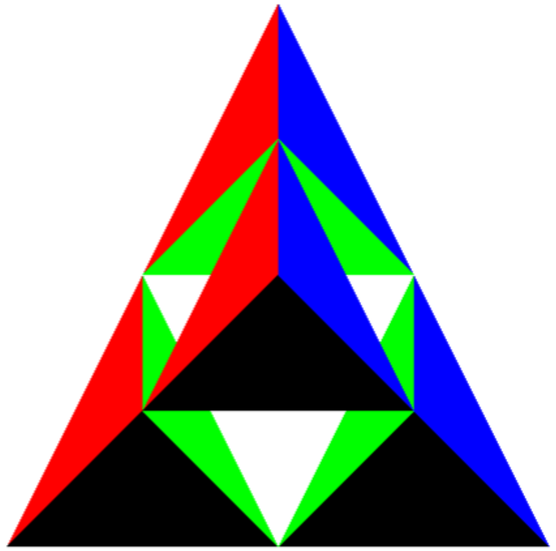
typedef float point[3];

/* initial tetrahedron */
point v[4] =  {  {0.0, 0.0, 10.0},  {0.0, 10.0, -10.0},
              {-10.0, -10.0, -10.0}, {10.0, -10.0, -10.0}
              };
int n;

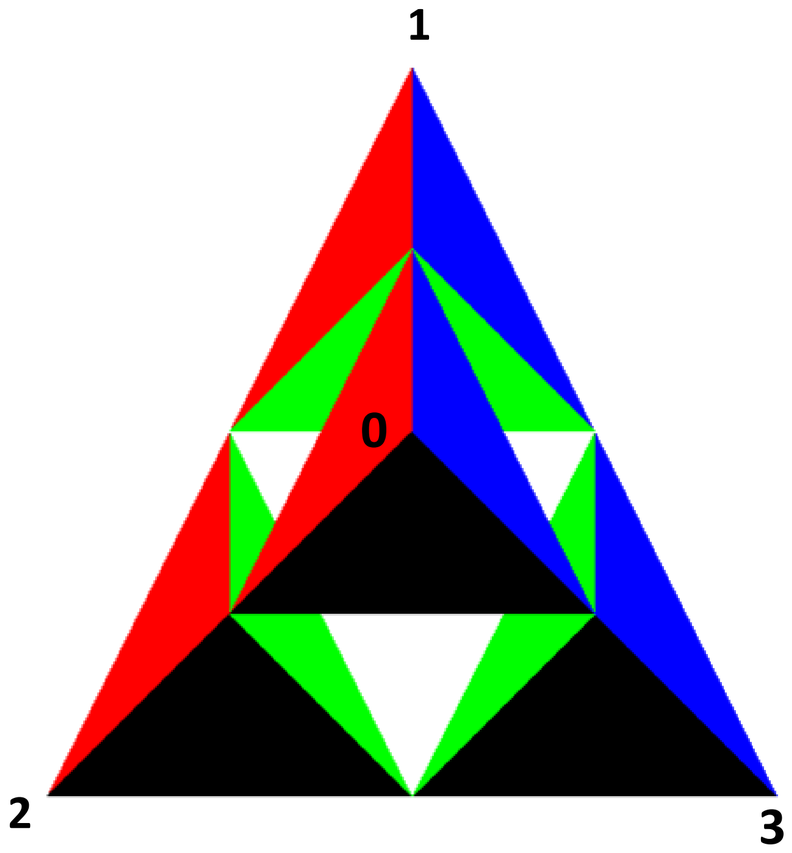
/* display one triangle */
void triangle( point a, point b, point c)
{      glBegin(GL_POLYGON);
              glVertex3fv(a);
              glVertex3fv(b);
              glVertex3fv(c);
       glEnd();
}
```

```
/* triangle subdivision */
void divide_triangle(point a, point b, point c, int m)
{
        point p1, p2, p3;
        int i;
        if(m>0)
        {
           for(i=0; i<3; i++) p1[i]=(a[i]+b[i])/2;
           for(i=0; i<3; i++) p2[i]=(a[i]+c[i])/2;
           for(i=0; i<3; i++) p3[i]=(b[i]+c[i])/2;

           divide_triangle(a, p1, p2, m-1);
           divide_triangle(c, p2, p3, m-1);
           divide_triangle(b, p3, p1, m-1);
        }
        else
        triangle(a,b,c); /* draw triangle at end of recursion */
}
```



```
/* Apply triangle subdivision to faces of tetrahedron */
void tetrahedron( int m)
{
        glColor3f(1.0,0.0,0.0); //anti clockwise-visible
        divide_triangle(v[0], v[1], v[2], m);
        glColor3f(0.0,1.0,0.0); //clockwise-not visible surface
        divide_triangle(v[3], v[2], v[1], m);
        glColor3f(0.0,0.0,1.0);
         divide_triangle(v[0], v[3], v[1], m);
        glColor3f(0.0,0.0,0.0);
        divide_triangle(v[0], v[2], v[3], m);
}
```



```
void display()
{
   glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
   // to reset the buffer  and for hidden surface removal
   glLoadIdentity(); // loading identity matrix
   tetrahedron(n);
   glFlush();
}

void myReshape(int w, int h) // zoom in and zoom out
{
   glViewport(0, 0, w, h); // to set the viewport according to changed w and h
   glMatrixMode(GL_PROJECTION);
   // matrix mode – defines CTM, GL_PROJECTION  to view in ortho
   glLoadIdentity(); // identity matrix resets the matrix to its default state
   if (w <= h)
   glOrtho(-12.0, 12.0, -12.0 * (GLfloat) h / (GLfloat) w, 12.0 * (GLfloat) h / (GLfloat) w, -12.0, 12.0);
   else
   glOrtho(-12.0 * (GLfloat) w / (GLfloat) h, 12.0 * (GLfloat) w / (GLfloat) h, -12.0, 12.0, -12.0, 12.0);
   glMatrixMode(GL_MODELVIEW); // transformations done in model view
   glutPostRedisplay();
}
```

```
void main(int argc, char **argv)
{
        printf(" No. of Divisions ? ");
        scanf("%d",&n);
        glutInit(&argc, argv);
        glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB | GLUT_DEPTH);
        /* This is to view the behind triangle */
        glutInitWindowSize(500, 500);
        glutCreateWindow("3D Gasket");
        glClearColor (1.0, 1.0, 1.0, 1.0);
        glutReshapeFunc(myReshape);
        glutDisplayFunc(display);
        glEnable(GL_DEPTH_TEST);  // enable the hidden surface
        glutMainLoop();
}
```

Display Lists

- Conceptually similar to a graphics file
  - Must define (name, create)
  - Add contents
  - Close
- In client-server environment, display list is placed on server
  - Can be redisplayed without sending primitives over network each time

Display List Functions

- Creating a display list

```
GLuint id;

void init()
{
   id = glGenLists( 1 );
   glNewList( id, GL_COMPILE );
   /* other OpenGL routines */
   glEndList();
}
```


- Call a created list

```
void display()
{
   glCallList( id );
}
```

Display Lists and State

- Most OpenGL functions can be put in display lists
- State changes made inside a display list persist after the display list is executed
- Can avoid unexpected results by using **`glPushAttrib`** and **`glPushMatrix`** upon entering a display list and **`glPopAttrib`** and **`glPopMatrix`** before exiting

VIRTUAL REALITY (VR)

VIRTUAL REALITY (VR)

Introduction • Three I's • Commercial Technology • Five Classic Components

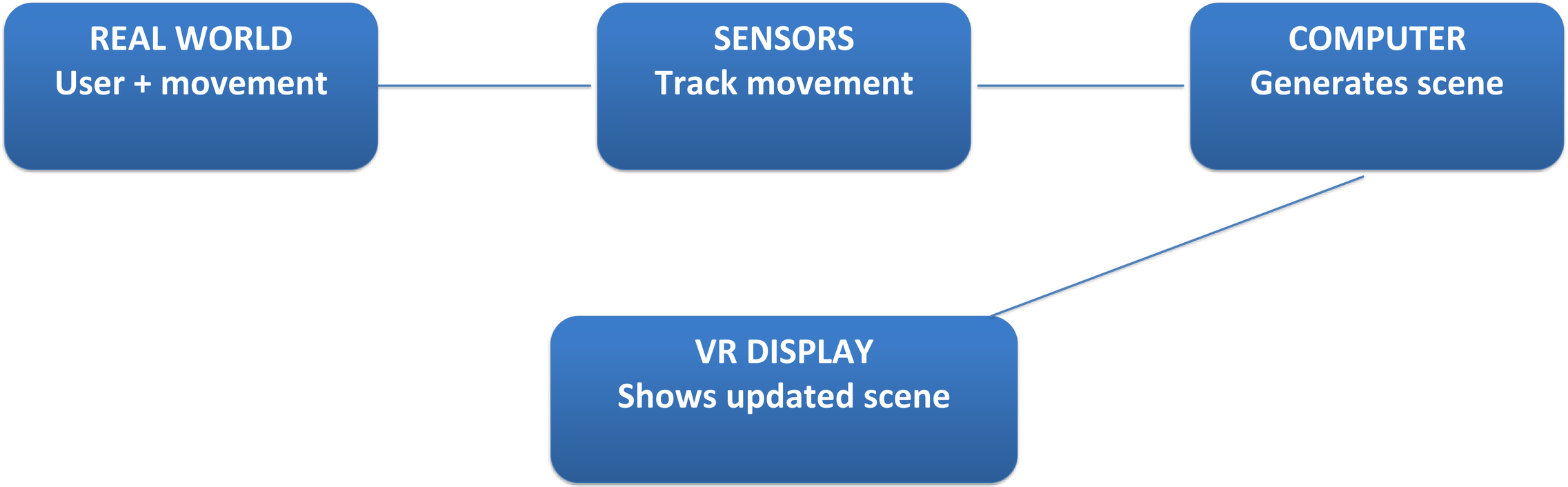


1. What is Virtual Reality?

- Virtual Reality (VR) is a computer-created environment that makes a person feel as if they are inside another world.
- A VR headset shows computer-generated images to our eyes.
- Sensors track movements such as head turning and hand movement.
- The virtual scene changes according to our movements.



Real World → Virtual World



Continuous loop: Move → Sense → Compute → Display → User sees the updated virtual world

Applications of VR

EDUCATION

Virtual laboratories
3D learning

MEDICAL

Training and simulation
Surgical practice

GAMES

Immersive games
Interactive worlds

ENGINEERING

Design review
Virtual prototypes

TRAINING

Flight/driving
Safety simulation

ARCHITECTURE

Walk through buildings
Before construction


Sun
Earth
Mars
VR Classroom


Tasks
Explore Anatomy
Identify Structures
Perform Procedure
Complete Simulation
Medical Simulator


Level 3
Score: 1250
VR Gaming


Living Room
Kitchen
Bedroom
Bathroom
Exterior
Virtual Building Walkthrough

2. The Three I's of Virtual Reality

Three important ideas that describe the VR experience

IMMERSION
Feeling of being inside the virtual world

INTERACTION
Ability to act on and control the world

IMAGINATION
Ability to experience things beyond reality

Memory trick: I = Immersion + Interaction + Imagination

Three I's — 1. Immersion

- Immersion means the user feels surrounded by the virtual environment.
- The goal is to reduce awareness of the real world.
- A headset can provide a wide visual field and stereoscopic 3D images.
- Head tracking makes the scene change when the user turns the head.
- Example: Looking around a virtual museum and feeling present inside it.



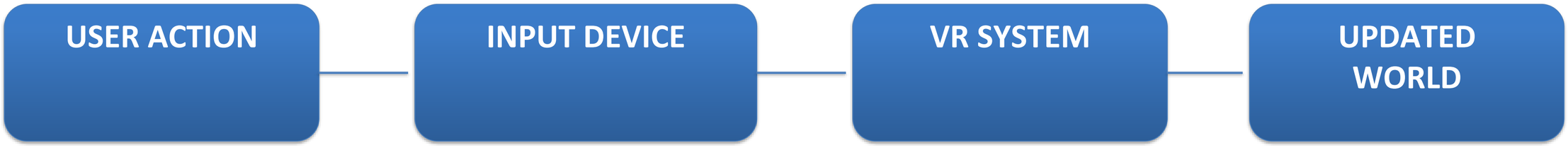
Three I's — 2. Interaction

Interaction means the user can do something in the virtual world.

Controllers, hand tracking, motion sensors and other input devices can be used.

The system detects the user's action and updates the virtual scene.

Example: Pointing at a virtual object and moving it with a controller.



Three I's — 3. Imagination

Imagination is the ability to experience computer-created situations that may not exist in the real world.

VR can create places, objects and experiences that are difficult, expensive or impossible to create physically.

Example: Explore Mars, travel through the human body, or visit a historical place virtually.

The computer gives the user a believable environment for the imagined experience.

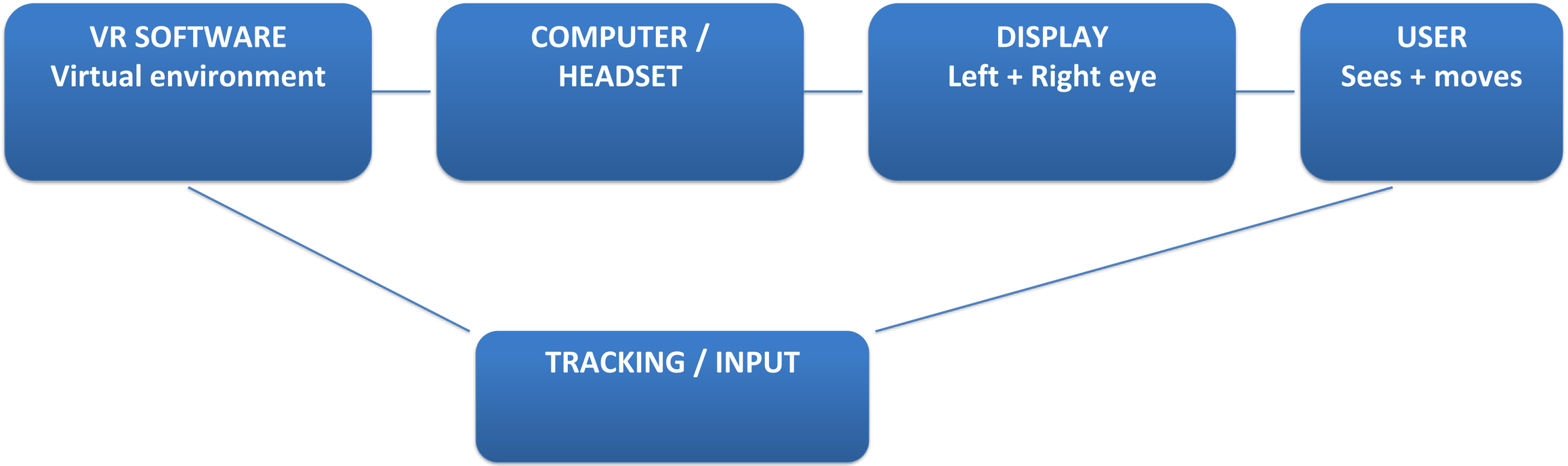


3. Commercial VR Technology

- Commercial VR means VR technology made available as products for users, businesses and organizations.
- VR systems commonly combine a headset/display, motion tracking, controllers and software.
- Standalone VR headsets contain computing hardware inside the headset.
- PC-based VR systems use a computer to generate the virtual environment.
- Some systems use external sensors or cameras for tracking.
- Common commercial uses inc ducation, design and simulation.



Commercial VR — Basic System Flow



The loop keeps repeating so the virtual world responds to the user's movement.

4. Five Classic Components of a VR System

1
VR Engine

Creates the virtual world

2
Task

Shows the virtual world

3
user

Detects user movement

4
I/O Devices

Lets the user interact

5
Software & Databases

Processes data and updates the scene

These components work together to create the VR experience.

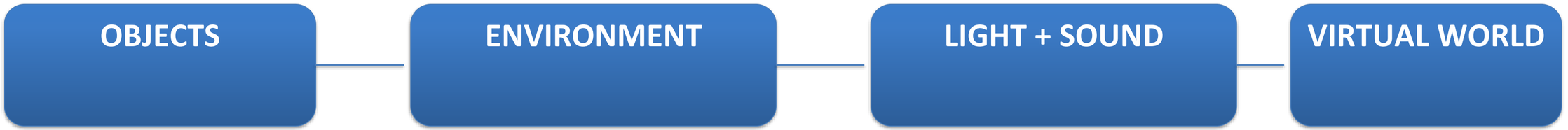
VR Engine

Manages the position, movement, and behavior of objects in the virtual world.

Processes user interactions such as touching, grabbing, or moving virtual objects.

Updates the virtual scene continuously according to the user's actions.

Provides realistic effects such as shadows, reflections, animations, and object responses..



Task

Creates the virtual environment and its overall scene.

Defines virtual objects, surfaces, lighting, textures, and sounds.

Controls how objects behave and interact within the virtual world.

Generates the scene in real time based on the user’s actions and movements.



User

Experiences the virtual environment through VR devices.

Views and explores the virtual world from different directions.

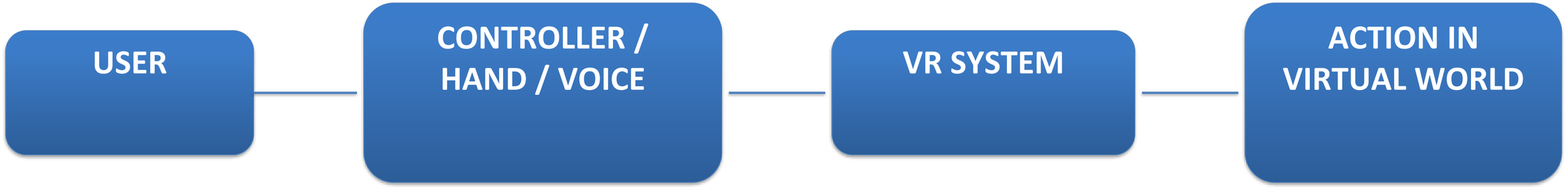
Interacts with virtual objects using controllers, hands, or other input devices.

Performs actions and receives visual, audio, and other feedback from the VR system.



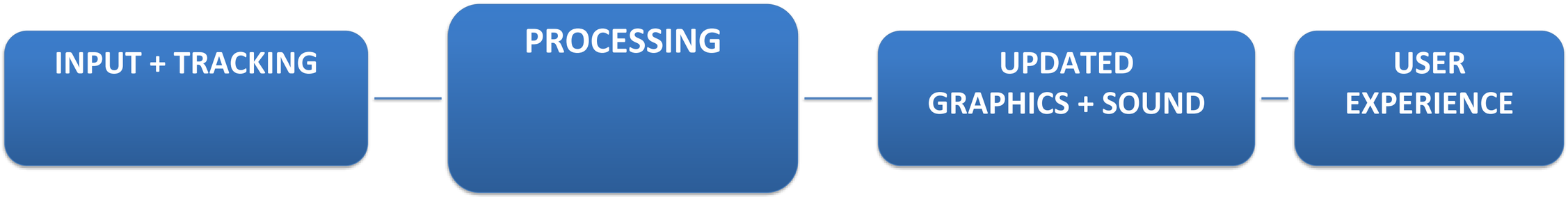
Input/Output Devices

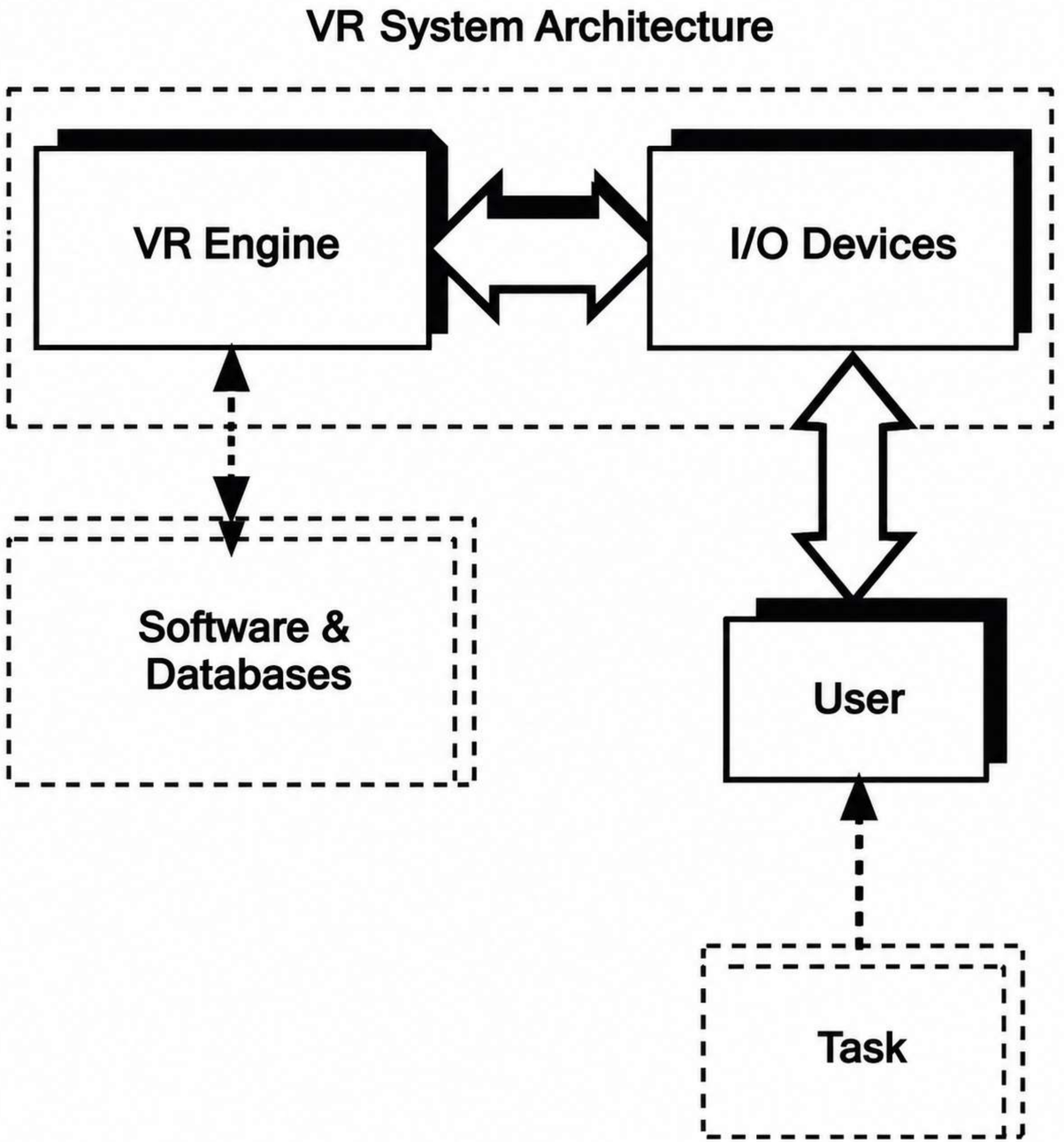
- Input devices allow the user to send commands to the VR system.
- Examples: handheld controllers, buttons, hand tracking, voice input and other sensors.
- Input can be used to select, move, grab or control virtual objects.
- Simple idea: Input = “What does the user want to do?”



Software & Databases

- The computer processes input and tracking information.
- It calculates what the user should see next.
- It updates graphics, sound, physics and interactions.
- Fast processing is important because the virtual world should respond quickly to user movement.




VR System Architecture
VR Engine
I/O Devices
Software &
Databases
User
Task